PENGEMBANGAN FITUR DASHBOARD PADA SISTEM PROJECT MANAGEMENT BERDASARKAN DATA DAILY REPORT

(Studi Kasus : PT Smart Home Inovasi Yogyakarta)

LAPORAN TUGAS AKHIR

DIAN PUTRI ISWANDI

5220311118

PROGRAM STUDI SISTEM INFORMASI PROGRAM SARJANA

FAKULTAS SAINS & TEKNOLOGI

UNIVERSITAS TEKNOLOGI YOGYAKARTA

2026

PENGEMBANGAN FITUR DASHBOARD PADA SISTEM PROJECT MANAGEMENT BERDASARKAN DATA DAILY REPORT

(Studi Kasus : PT Smart Home Inovasi Yogyakarta)

Disusun oleh

DIAN PUTRI ISWANDI

5220311118

Telah dipertahankan di depan Dewan Penguji
pada tanggal ……

DEWAN PENGUJI

Nama & Gelar	Jabatan	Tanda Tangan	Tanggal
…….		…….	
NIK.	Ketua Penguji		……..…
…….		…….	
NIK.	Penguji I		……..…
…….	Penguji II	…….	
NIK.	(Dosen Pembimbing)		……..…

Yogyakarta, ……
Ketua Program Studi Sistem Informasi

……

NIK.

LEMBAR PERNYATAAN

SURAT KETERANGAN TUGAS AKHIR

KATA PENGANTAR

ABSTRAK

ABSTRACT

DAFTAR ISI

DAFTAR TABEL

DAFTAR GAMBAR

BAB I
PENDAHULUAN

1.1 Latar Belakang

PT Smart Home Inovasi Yogyakarta (SHI), yang berlokasi di Kabupaten Sleman, Daerah Istimewa Yogyakarta, merupakan perusahaan teknologi yang mengkhususkan diri pada pengembangan solusi Smart Home dan *Internet of Things* (IoT). Mengusung visi sebagai penyedia solusi hunian cerdas terdepan, SHI menghadirkan inovasi perangkat otomasi—mulai dari sistem keamanan, pengaturan suhu, hingga efisiensi energi—guna menciptakan ekosistem hunian yang aman, nyaman, dan hemat energi. Secara operasional, proses bisnis perusahaan dimulai dari penerimaan pesanan klien, dilanjutkan dengan survei lokasi, perencanaan desain sistem, pelaksanaan instalasi perangkat, integrasi antar-komponen, hingga pengujian dan serah terima proyek kepada pelanggan. Untuk menunjang kegiatan operasional tersebut, SHI telah mengimplementasikan sebuah aplikasi manajemen proyek yang mencakup fungsi pengelolaan data proyek, penjadwalan, penugasan teknisi, serta pencatatan laporan harian (*daily report*) sebagai acuan utama dalam memantau kemajuan fisik dan kondisi aktual proyek di lapangan.

Meskipun aplikasi manajemen proyek memiliki fitur daily report, sistem hanya dapat diakses manajer sehingga teknisi lapangan harus melaporkan secara tidak langsung. Selain itu, tidak ada visualisasi dashboard untuk ringkasan kondisi seluruh proyek. Akibatnya, tanpa indikator risiko terukur, manajer sering mengabaikan proyek kritis dan deteksi masalah menjadi lambat (reactive). Hal ini berdampak pada penurunan efisiensi operasional dan kepuasan pelanggan.

Berdasarkan permasalahan tersebut, penelitian ini menawarkan solusi berupa pengembangan fitur dashboard pada aplikasi manajemen proyek yang telah berjalan di PT SHI. Pengembangan ini mencakup dua aspek utama, yaitu pemberian akses sistem kepada teknisi lapangan untuk menginputkan data daily report secara langsung, serta pembuatan fitur dashboard dengan kemampuan *smart system*. Fitur dashboard yang

dikembangkan memiliki kemampuan otomasi berupa kategorisasi status kesehatan proyek (Project Health Status) yang bekerja dengan cara sistem secara otomatis menghitung persentase progres ideal berdasarkan posisi hari berjalan terhadap total durasi proyek, kemudian membandingkannya dengan persentase realisasi aktual yang diinputkan oleh teknisi. Hasil perbandingan tersebut secara otomatis divisualisasikan dalam bentuk indikator kode warna pada tampilan dashboard, sehingga manajer dapat mengidentifikasi kondisi kesehatan setiap proyek secara sekilas. Pendekatan serupa telah dibuktikan efektivitasnya oleh Ernawan (2024) dalam penelitiannya pada perusahaan konstruksi, di mana implementasi dashboard manajemen mampu membantu eksekutif memantau indikator kinerja proyek secara real-time serta meningkatkan akurasi data untuk keperluan pemantauan.

Melalui implementasi fitur dashboard ini, diharapkan alur pelaporan dari teknisi lapangan ke manajer menjadi lebih lancar karena teknisi dapat menginputkan data daily report secara langsung ke dalam sistem tanpa perantara. Dengan visualisasi status kesehatan proyek, manajer dapat melakukan deteksi dini terhadap kondisi keterlambatan secara sekilas tanpa harus memeriksa laporan satu per satu. Secara keseluruhan, pengembangan ini diharapkan dapat membantu PT SHI memantau jalannya proyek dengan lebih efisien serta meminimalisir risiko keterlambatan penyelesaian proyek.

1.2 Rumusan Masalah

Bagaimana merancang fitur *dashboard* yang mampu mengolah data *daily report* menjadi visualisasi status proyek yang mudah dipahami guna membantu manajer proyek dalam menentukan prioritas penanganan masalah secara objektif di PT Smart Home Inovasi Yogyakarta?

1.3 Batasan Masalah

1. Data yang diolah dalam sistem terbatas pada data *daily report* yang mencakup nama proyek, tanggal laporan, persentase progres harian, dan catatan kendala, serta data *baseline* jadwal proyek (tanggal mulai dan target selesai).

2. Sistem menghitung deviasi progres dengan membandingkan persentase realisasi aktual dari *daily report* terhadap progres ideal berdasarkan *baseline* jadwal, tanpa menggunakan metode prediksi berbasis *machine learning* atau kecerdasan buatan.
3. Sistem secara otomatis mengkategorikan status kesehatan proyek (*Project Health Status*) ke dalam beberapa kategori berdasarkan nilai deviasi, serta mengurutkan proyek berdasarkan tingkat urgensi pada tampilan dashboard.
4. *Output* sistem berupa fitur dashboard yang menampilkan ringkasan proyek aktif dengan indikator kode warna (hijau, kuning, merah), serta detail progres aktual, progres ideal, deviasi, dan catatan kendala terakhir.
5. Pengguna sistem terdiri dari Manajer Proyek sebagai pemantau dashboard dan Teknisi Lapangan sebagai penginput data *daily report*.
6. Sistem tidak mencakup pengelolaan data di luar lingkup *daily report*, seperti data keuangan, pengadaan material, atau manajemen sumber daya manusia.

1.4 Tujuan Penelitian

Merancang dan mengembangkan fitur dashboard pada sistem *project management* yang mampu mengolah data *daily report* secara otomatis menjadi indikator visual status kesehatan proyek (*Project Health Status*) di PT SHI.

1.5 Manfaat Penelitian

1. Meningkatkan objektivitas pengambilan keputusan di PT SHI, di mana manajer proyek dapat menentukan prioritas penanganan proyek berdasarkan indikator visual yang terukur, bukan lagi berdasarkan intuisi atau penilaian subjektif.
2. Mempercepat identifikasi proyek bermasalah di PT SHI, karena sistem secara otomatis menonjolkan proyek dengan status kritis ke urutan teratas dashboard, sehingga potensi keterlambatan fatal pada proyek instalasi dan integrasi sistem *Smart Home* dapat dideteksi lebih dini dan ditangani secara preventif.

1.6 Sistematika Penulisan

BAB I PENDAHULUAN

Bab ini menguraikan latar belakang masalah yang mendasari penelitian, rumusan masalah, batasan masalah yang menjadi lingkup pengembangan

sistem, tujuan penelitian, manfaat penelitian bagi PT SHI, serta sistematika penulisan laporan.

BAB II KAJIAN HASIL PENELITIAN DAN LANDASAN TEORI

Bab ini memuat kajian hasil penelitian terdahulu yang relevan dengan topik pengembangan dashboard dan sistem manajemen proyek sebagai referensi perbandingan, serta landasan teori yang digunakan sebagai dasar dalam perancangan dan pengembangan sistem.

BAB III METODE PENELITIAN

Bab ini menjelaskan metode penelitian yang digunakan dalam proses pengembangan sistem, termasuk tahapan-tahapan pengembangan perangkat lunak yang diterapkan, serta gambaran obyek penelitian yaitu PT SHI.

BAB IV ANALISIS DAN PERANCANGAN

Bab ini membahas analisis kebutuhan sistem serta perancangan sistem yang meliputi perancangan model proses, perancangan model data, perancangan fisik basis data, perancangan relasi antar tabel, dan perancangan antarmuka pengguna (*user interface*).

BAB V IMPLEMENTASI DAN PEMBAHASAN SISTEM

Bab ini menyajikan hasil implementasi sistem yang mencakup implementasi basis data dan implementasi antarmuka, dilanjutkan dengan pembahasan hasil sistem yang meliputi pembahasan basis data, pembahasan fungsionalitas sistem, pengujian sistem, serta inovasi sistem yang dikembangkan.

BAB VI PENUTUP

Bab ini berisi kesimpulan dari keseluruhan hasil penelitian yang menjawab rumusan masalah serta saran untuk pengembangan sistem di masa mendatang.

BAB II
KAJIAN HASIL PENELITIAN DAN LANDASAN TEORI

2.1 Kajian Hasil Penelitian

Kajian hasil penelitian ini memuat tinjauan terhadap beberapa penelitian terdahulu yang relevan dengan topik pengembangan fitur dashboard pada sistem *project management*. Penelitian-penelitian tersebut dijadikan sebagai bahan referensi dan perbandingan untuk memahami pendekatan, metode, serta hasil yang telah dicapai oleh peneliti sebelumnya. Melalui kajian ini, dapat diidentifikasi persamaan dan perbedaan antara penelitian terdahulu dengan penelitian yang dilakukan penulis, sehingga dapat diketahui posisi dan kontribusi penelitian ini terhadap pengembangan keilmuan di bidang sistem informasi manajemen proyek.

Penelitian yang dilakukan oleh Azkia dkk. (2024) dengan judul "*Perancangan Dashboard Monitoring dan Controlling Kinerja Proyek pada PT. XYZ Menggunakan Metode Agile and Lean Development*" menyoroti permasalahan keterlambatan dan pembengkakan anggaran pada proyek konstruksi telekomunikasi yang disebabkan oleh tidak adanya alat *monitoring* dan kontrol informasi yang efektif. Penelitian ini bertujuan merancang *dashboard monitoring* dan *controlling* kinerja proyek menggunakan metode *Agile and Lean Development* dengan mengintegrasikan konsep *Earned Value Management* (EVM) sebagai alat pengukuran kinerja. Dashboard yang dihasilkan mencakup tujuh fitur utama yaitu *Project Overview, Financial Tracking, Timeline, Milestone, Resource, Performance,* dan *Risk*, yang dikembangkan menggunakan *Google Looker Studio* dengan sumber data dari *Google Sheets*. Hasil pengujian melalui *Black Box Testing* menunjukkan seluruh fitur berfungsi sesuai spesifikasi, sementara *User Acceptance Test* (UAT) memperoleh skor 86% yang termasuk kategori sangat baik. Perbedaan utama dengan penelitian proyek tugas akhir ini terletak pada cakupan dan kompleksitas dashboard, di mana penelitian Azkia dkk. mencakup aspek yang lebih luas meliputi keuangan, sumber daya, dan manajemen risiko dengan metode EVM, sementara penelitian ini berfokus secara spesifik pada

pengembangan fitur dashboard berbasis data *daily report* dengan mekanisme otomasi kategorisasi status kesehatan proyek (*Project Health Status*) menggunakan perhitungan deviasi progres sederhana.

Penelitian yang dilakukan oleh Luthan dkk. (2023) dengan judul "*Reinventing Formulas for Construction Project Delay Index Due to Management and Production*" menyoroti permasalahan ketidakadilan dalam pembebanan sanksi keterlambatan proyek konstruksi, di mana selama ini pihak produksi (kontraktor) selalu menjadi satu-satunya pihak yang dikenai denda, sementara pihak manajemen (konsultan) tidak pernah dikenai sanksi meskipun turut berkontribusi terhadap keterlambatan. Penelitian ini bertujuan membangun formula yang akurat untuk menghitung *Management Delay Index* (MDI) dan *Production Delay Index* (PDI) yang diintegrasikan ke dalam dashboard *Ms. Project*. Hasil penelitian menunjukkan bahwa formula yang diintegrasikan ke dashboard mampu menyajikan informasi yang lebih rinci dibandingkan perhitungan manual, meliputi nilai kontrak, progres aktual, progres rencana, deviasi progres, penyebab keterlambatan, serta indeks keterlambatan dari aspek manajemen dan produksi. Uji kelayakan menggunakan metode TELOS memperoleh skor rata-rata 8,26 dari skala 10 yang menunjukkan formula tersebut sangat layak dan akurat. Perbedaan utama dengan penelitian proyek tugas akhir ini terletak pada fokus dan kompleksitas formula, di mana penelitian Luthan dkk. mengembangkan formula indeks keterlambatan yang melibatkan variabel *free float*, *pending*, waktu mulai aktual, dan waktu selesai aktual untuk menentukan pihak yang bertanggung jawab atas keterlambatan, sementara penelitian ini menggunakan perhitungan deviasi progres sederhana berupa selisih antara progres ideal dan progres realisasi dari data *daily report* untuk mengkategorikan status kesehatan proyek ke dalam indikator kode warna pada dashboard.

Penelitian yang dilakukan oleh Ernawan (2024) dengan judul "*Pemanfaatan Management Dashboard dalam Pengambilan Keputusan Strategis pada Perusahaan Bisnis Konstruksi (Studi Kasus PT. XYZ)*" menyoroti permasalahan kurang optimalnya

pengambilan keputusan strategis akibat keterbatasan akses data kinerja proyek yang akurat dan *real-time*. Penelitian ini bertujuan menganalisis peran dashboard manajemen sebagai sistem pendukung keputusan strategis dalam memantau indikator kinerja utama (KPI) perusahaan konstruksi. Hasil penelitian menunjukkan bahwa dashboard manajemen mampu meningkatkan akurasi data, mempercepat identifikasi masalah proyek, serta memperkuat koordinasi lintas divisi. Perbedaan utama dengan penelitian proyek tugas akhir ini adalah penelitian Ernawan berorientasi pada evaluasi dashboard sebagai alat pengambilan keputusan strategis tingkat perusahaan, sementara penelitian ini menekankan pengembangan fitur dashboard operasional berbasis *daily report* dalam sistem *project management*.

Penelitian yang dilakukan oleh Gledson dkk. (2024) dengan judul "Reporting on the Development of a Web-Based Prototype Dashboard for Construction Design Managers, Achieved through Design Science Research Methodology (DSRM)" menyoroti permasalahan inefisiensi dalam proses manajemen desain konstruksi, khususnya terkait koordinasi yang terfragmentasi, pemantauan kinerja desainer yang tidak memadai, serta pengelolaan arus informasi yang masih bersifat manual dan "analogue". Penelitian ini bertujuan mengembangkan purwarupa dashboard manajemen desain berbasis web untuk mendigitalisasi proses-proses tersebut dengan fokus pada pemantauan koordinasi desain, prioritas tugas, dan pelacakan kinerja desainer melalui analisis data Technical Queries (TQs) dan Requests for Information (RFIs). Hasil penelitian menunjukkan bahwa dashboard yang dikembangkan mampu meningkatkan efisiensi dan produktivitas dengan menyediakan visualisasi data real-time, seperti analisis RFI, pemantauan jadwal produksi desain (Information Release Schedule), serta tren kinerja desainer di tingkat proyek maupun portofolio. Umpan balik dari praktisi mengonfirmasi bahwa fitur-fitur tersebut memberikan manfaat signifikan dalam peran manajemen desain. Perbedaan utama dengan penelitian tugas akhir ini terletak pada fokus dan pengguna akhir dashboard. Penelitian Gledson dkk. berfokus pada dukungan keputusan untuk Manajer Desain dengan memanfaatkan data dari proses desain (seperti TQ dan RFI) untuk

mengoptimalkan koordinasi dan alur kerja desain, sementara penelitian ini berfokus pada pemantauan kesehatan proyek secara umum dari sudut pandang kemajuan waktu dengan menggunakan data daily report dari teknisi lapangan. Perhitungan yang digunakan juga berbeda, di mana penelitian Gledson dkk. mengandalkan analisis tren dan beban kerja berdasarkan data kueri teknis, sedangkan penelitian ini menggunakan perhitungan deviasi progres sederhana berupa selisih antara progres ideal (berdasarkan durasi proyek) dan progres realisasi dari daily report untuk mengkategorikan status kesehatan proyek ke dalam indikator kode warna pada dashboard.

Penelitian yang dilakukan oleh Hakim dan Pradibta (2025) dengan judul "Pengembangan Fitur Dashboard dan Report pada Aplikasi Project Management PT Intelix untuk Optimalisasi Pemantauan Proyek" membahas pengembangan fitur dashboard dan report pada aplikasi manajemen proyek di PT Intelix untuk mengatasi permasalahan pemantauan proyek yang tidak terintegrasi dan pembuatan laporan manual menggunakan Microsoft Excel. Menggunakan metode Agile Kanban, penelitian ini menambahkan tiga tabel proyek (High Priority Project, Milestone Project, dan Implementation Project) pada menu Dashboard serta beberapa sub-menu pada menu Report. Hasil pengujian menunjukkan seluruh fungsi berjalan baik, mendapatkan respons "sangat sesuai" dari pengguna, dan sistem dinilai baik melalui evaluasi SUS. Penelitian Hakim & Pradiba mendukung penelitian ini karena sama-sama membuktikan bahwa pengembangan fitur dashboard mampu menyajikan informasi proyek secara lebih terperinci dan memudahkan pemantauan progres proyek secara real-time.

Penelitian yang dilakukan oleh Auliansyah dkk. (2023) dengan judul "*Rancang Bangun Sistem Monitoring Manajemen Proyek Konstruksi Menggunakan Kurva-S*" membahas pengembangan sistem *monitoring* berbasis website untuk proyek konstruksi di CV. Cahya Grafika menggunakan metode Kurva-S. Penelitian ini dilatarbelakangi oleh permasalahan keterbatasan sumber daya perusahaan dalam mengelola banyak proyek konstruksi secara simultan, serta kebutuhan untuk mengontrol kesesuaian antara rencana dan realisasi pekerjaan guna menghindari keterlambatan proyek. Tujuan

penelitian ini adalah membangun sistem yang dapat membuat rencana anggaran biaya (RAB), *schedule* rencana, laporan mingguan proyek, dan rekapitulasi laporan proyek dengan *output* berupa Gantt Chart dan Kurva-S yang menampilkan perbandingan antara kurva rencana dan kurva realisasi. Sistem yang dikembangkan memiliki fitur *multi-user* dengan empat hak akses berbeda (admin, manager proyek, manager lapangan, dan direktur). Hasil penelitian menunjukkan bahwa sistem berhasil menghitung nilai deviasi keterlambatan proyek, dengan studi kasus pada proyek peningkatan jalan Wadungasri-Tambaksumur menunjukkan nilai deviasi -0,17% pada minggu ke-1 dan -1,19% pada minggu ke-2 yang mengindikasikan keterlambatan. Pengujian *black-box* pada seluruh hak akses menunjukkan sistem berfungsi sesuai harapan. Penelitian Auliansyah dkk. mendukung penelitian ini karena sama-sama mengembangkan sistem *monitoring* proyek dengan membandingkan progres rencana dan progres aktual, serta menggunakan perhitungan deviasi untuk mendeteksi keterlambatan proyek. Namun demikian, penelitian ini juga menjadi pembanding karena terdapat perbedaan pada metode dan fokus pengembangan. Penelitian Auliansyah dkk. (2023) menggunakan metode Kurva-S yang memerlukan perhitungan bobot kontrak dan bobot rencana komulatif yang kompleks, serta mencakup fitur RAB, Gantt Chart, dan *multi-user* dengan empat level akses. Sementara itu, penelitian ini lebih sederhana dengan fokus pada pengembangan *smart system* berupa kategorisasi otomatis status kesehatan proyek menggunakan perhitungan deviasi sederhana antara progres ideal (berdasarkan durasi proyek) dan progres aktual dari data *daily report* yang diinputkan langsung oleh teknisi lapangan, yang kemudian divisualisasikan dalam bentuk indikator kode warna untuk memudahkan deteksi dini proyek bermasalah.

Penelitian yang dilakukan oleh Iqbal dkk. (2024) dengan judul "*Sistem Manajemen Proyek pada Startup Jasa Pembuatan Aplikasi*" membahas perancangan dan pengembangan sistem manajemen proyek berbasis web untuk penyedia layanan pengembangan aplikasi di lingkungan *startup* teknologi. Penelitian ini dilatarbelakangi oleh permasalahan umum yang sering muncul dalam manajemen proyek di *startup*,

seperti keterlambatan, pembengkakan biaya, dan kualitas produk di bawah standar akibat ketidakmampuan dalam mengatur proyek secara efisien. Tujuan penelitian ini adalah merancang sistem yang dapat memfasilitasi kerja tim, manajemen tugas, pelaporan waktu nyata (*real-time*), dan pelacakan kemajuan proyek. Metode pengembangan yang digunakan adalah Waterfall dengan tahapan *communication*, *planning*, *modelling*, *construction*, dan *deployment*. Hasil penelitian menunjukkan bahwa sistem yang dikembangkan mampu mengelola berbagai aspek proyek meliputi manajemen tugas, kolaborasi tim, pelacakan anggaran, serta menampilkan *dashboard* ringkasan proyek, *Gantt chart*, dan log aktivitas. Pengujian menggunakan metode *Black Box* pada seluruh fitur menunjukkan sistem beroperasi dengan stabilitas baik dan sesuai harapan pengguna. Penelitian ini mendukung penelitian tugas akhir karena sama-sama mengembangkan sistem manajemen proyek berbasis web yang memudahkan pemantauan proyek melalui *dashboard* dengan ringkasan informasi proyek, progres tugas, dan kolaborasi tim. Namun demikian, penelitian ini juga menjadi pembanding karena terdapat perbedaan pada fokus dan mekanisme pemantauan. Penelitian Iqbal dkk. (2024) berfokus pada pengembangan sistem manajemen proyek komprehensif untuk *startup* jasa pembuatan aplikasi dengan cakupan fitur yang luas meliputi manajemen tugas, tim, anggaran, serta *Gantt chart*, dan menggunakan metode Waterfall dalam pengembangannya. Sementara itu, penelitian ini lebih spesifik pada pengembangan fitur *dashboard* dengan kemampuan *smart system* berupa kategorisasi otomatis status kesehatan proyek menggunakan perhitungan deviasi sederhana antara progres ideal (berdasarkan durasi proyek) dan progres aktual dari data *daily report* yang diinputkan langsung oleh teknisi lapangan, yang kemudian divisualisasikan dalam bentuk indikator kode warna untuk memudahkan deteksi dini proyek bermasalah tanpa mencakup fitur manajemen anggaran dan *Gantt chart* secara mendalam.

Tabel 2.1 Perbandingan Penelitian Terdahulu

No	Judul	Penulis	Tahun	Hasil
1	Perancangan Dashboard Monitoring dan Controlling Kinerja Proyek pada PT. XYZ Menggunakan Metode Agile and Lean Development	Rifan Azkia, Yusuf Nugroho Doyo Yekti, Dino Caesaron	2024	Menghasilkan dashboard dengan fitur *Project Overview*, *Financial Tracking*, *Timeline*, *Milestone*, *Logistic*, *Performance*, dan *Risk*. Hasil validasi menunjukkan dashboard relevan dengan tingkat kepuasan pengguna 86%.
2	Pemanfaatan Management Dashboard dalam Pengambilan Keputusan Strategis pada Perusahaan Bisnis Konstruksi (Studi Kasus PT. XYZ)	Ernawan	2024	Dashboard manajemen membantu eksekutif memantau KPI secara *real-time*, meningkatkan akurasi data, memfasilitasi komunikasi antar departemen, serta memberikan dampak positif pada biaya, mutu, dan waktu.
3	Reporting on the Development of a Web-Based Prototype Dashboard for Construction Design Managers	Barry Gledson, Kay Rogage, Anna Thompson, Hazel Ponton	2024	Mengembangkan purwarupa *dashboard* berbasis web untuk manajer desain konstruksi dengan fokus pada pemantauan koordinasi desain, prioritas tugas, dan pelacakan kinerja desainer melalui analisis data TQ dan RFI.
4	Pengembangan Fitur Dashboard dan Report pada Aplikasi Project Management PT Intelix untuk Optimalisasi Pemantauan Proyek	Afif Lukmanul Hakim, Hendra Pradibta	2025	Menambahkan fitur *High Priority Project*, *Milestone Project*, dan *Implementation Project* pada menu *Dashboard* serta sub-menu pada menu *Report*. Pengujian Black Box dan UAT menunjukkan seluruh fungsi berjalan baik dengan respons "sangat sesuai".
5	Rancang Bangun Sistem Monitoring Manajemen Proyek Konstruksi Menggunakan Kurva-S	Cindia Rama Auliansyah, Joseph Dedy Irawan, Fransiscus Xaverius Ariwibisono	2023	Membangun sistem monitoring berbasis *website* dengan metode Kurva-S yang dapat membuat RAB, *schedule* rencana, laporan mingguan, dan rekapitulasi laporan dengan output *Gantt Chart* dan Kurva-S. Sistem berhasil menghitung deviasi keterlambatan proyek.

No	Judul	Penulis	Tahun	Hasil
6	Sistem Manajemen Proyek pada Startup Jasa Pembuatan Aplikasi	Muhammad Iqbal, Emilia Emilia, Zahra Ramadhani, Dempi Ariska, Sri Rahayu, Serly Oktarina	2024	Mengembangkan sistem manajemen proyek berbasis web dengan metode Waterfall yang memfasilitasi kerja tim, manajemen tugas, pelaporan *real-time*, dan pelacakan kemajuan proyek. Pengujian Black Box menunjukkan sistem beroperasi dengan stabilitas baik.
7	Reinventing Formulas for Construction Project Delay Index Due to Management and Production	Putri Lynna A. Luthan, Nathanael Sitanggang, Syafriandi Syafriandi	2023	Membangun formula indeks keterlambatan proyek konstruksi dari aspek manajemen dan produksi yang diintegrasikan ke dalam *dashboard* Ms. Project. Formula mampu menyajikan informasi nilai kontrak, progres aktual, progres rencana, deviasi progres, penyebab keterlambatan, serta indeks keterlambatan manajemen dan produksi.

2.2 Landasan Teori

Dalam pengerjaan penelitian Tugas Akhir ini penulis menggunakan beberapa landasan teori yang menjadi acuan diantaranya sebagai berikut:

2.2.1 Manajemen Proyek dan Sistem *Monitoring*

Manajemen proyek merupakan disiplin ilmu yang mencakup proses perencanaan, pelaksanaan, dan pengendalian untuk memastikan tujuan proyek tercapai sesuai batasan waktu, biaya, dan ruang lingkup. Dalam proses pengendalian, manajer memerlukan informasi akurat sebagai dasar evaluasi dan pengambilan keputusan. Sistem monitoring berperan sebagai salah satu instrumen yang menyediakan informasi tersebut melalui pelacakan data aktivitas lapangan (Chaerul dkk., 2021). Praktik konvensional yang menggunakan alat tidak terintegrasi seperti spreadsheet terpisah sering menghambat aliran informasi dan mempersulit proses pengendalian. Oleh karena itu, penerapan project management system terpadu menjadi solusi untuk mengintegrasikan seluruh proses manajemen proyek, mulai dari pencatatan data harian hingga penyajian informasi bagi manajemen. Pengembangan fitur seperti dashboard

berbasis data daily report bertujuan untuk mendukung efektivitas pengelolaan proyek secara menyeluruh, termasuk koordinasi tim, evaluasi kemajuan, dan pengambilan keputusan strategis.

2.2.2 *Dashboard* Manajemen Proyek

Dalam *project management system*, *dashboard* berfungsi sebagai antarmuka pengguna grafis yang memvisualisasikan data operasional secara terpusat. Berbeda dengan sistem analitik prediktif, *dashboard* memfasilitasi transparansi informasi dengan mengubah rekapitulasi data *daily report* tekstual yang diinput oleh teknisi lapangan menjadi representasi visual terstruktur, seperti grafik kemajuan fisik dan indikator ketepatan waktu terhadap rencana awal atau *baseline* (Silmina & Azmi, 2025). Melalui transformasi visual ini, manajer proyek dapat memahami kondisi portofolio proyek secara komprehensif, mengevaluasi kinerja operasional berdasarkan fakta aktual, serta merumuskan keputusan tindak lanjut dengan jauh lebih efisien tanpa harus membaca laporan secara manual (Reddy, 2025).

2.2.3 Sistem Informasi *Daily Report* Proyek

Sistem Informasi *Daily Report* Proyek merupakan sebuah modul esensial di dalam ekosistem Sistem Manajemen Proyek (*Project Management System*) yang berfungsi untuk merekam aktivitas operasional harian secara terstruktur. Dalam siklus pelaksanaan proyek, pelaporan harian bertindak sebagai sumber data utama yang mendokumentasikan rincian progres pekerjaan, alokasi waktu, status penyelesaian tugas, serta kendala teknis di lapangan. Integrasi pelaporan harian ke dalam sistem manajemen proyek bertujuan untuk melakukan sentralisasi data historis, memastikan bahwa seluruh rekam jejak operasional dapat dilacak (*traceable*) secara aktual dan terhindar dari risiko kehilangan atau ketidakkonsistenan data akibat pencatatan manual (Alawiyah dkk. 2022).

Untuk memaksimalkan nilai dari himpunan data laporan harian tersebut, diperlukan pengembangan fitur antarmuka visual berupa *dashboard* pemantauan (*monitoring*). Fitur *dashboard* ini secara teknis akan mengekstrak, mengolah, dan menerjemahkan data *daily report* yang mentah menjadi informasi visual yang

komprehensif, seperti grafik progres, metrik capaian, maupun rekapitulasi status proyek. Dengan terintegrasinya data *daily report* ke dalam fitur *dashboard* berbasis web, proses pengawasan administrasi dan evaluasi proyek oleh pihak manajemen menjadi jauh lebih efisien, terpusat, dan dapat dipantau secara langsung (*real-time*) berdasarkan data faktual dari lapangan.

2.2.4 *Project Health Status*

Status kesehatan proyek (*Project Health Status*) merupakan indikator evaluatif faktual yang membandingkan progres aktual di lapangan dengan rencana awal (*baseline*). Alih-alih menggunakan analitik prediktif yang kompleks, indikator ini berfungsi sebagai instrumen audit visual untuk mendeteksi penyimpangan jadwal sedini mungkin. Pada implementasi *dashboard*, status kesehatan direpresentasikan melalui metode *Red-Amber-Green* (RAG) yang secara otomatis mengagregasi data mentah dari *daily report*. Melalui visualisasi warna yang intuitif ini—seperti indikator "Hijau" untuk operasional yang linear dengan jadwal dan "Merah" untuk kondisi kritis—pihak manajemen memperoleh transparansi informasi tingkat tinggi guna memantau kelancaran proyek secara *real-time* tanpa harus menelusuri tumpukan laporan manual (Fonseca dkk., 2025).

2.2.5 *Early Warning System* (EWS) Berbasis *Schedule Performance Index*

Dalam manajemen proyek operasional yang dinamis, *Early Warning System* (EWS) atau Sistem Peringatan Dini merupakan mekanisme proaktif yang diintegrasikan ke dalam perangkat lunak pemantauan untuk mendeteksi potensi penyimpangan jadwal secara otomatis. Dalam ranah sistem informasi operasional, EWS tidak dirancang menggunakan algoritma analitik prediktif yang kompleks, melainkan mengandalkan kemampuan sistem dalam mengotomatisasi pemrosesan data faktual secara *real-time*. Tujuan utama implementasi EWS adalah memangkas jeda waktu (*information lag*) antara terjadinya kendala progres di lapangan dengan sampainya peringatan tersebut kepada manajer proyek, sehingga mitigasi keterlambatan dapat dilakukan sedini mungkin berdasarkan data yang valid.

Sebagai parameter pemicu (*trigger*), EWS pada sistem ini mengadopsi metrik *Schedule Performance Index* (SPI). SPI bertindak sebagai indikator matematis pasti yang dihitung menggunakan rasio perbandingan antara persentase pekerjaan aktual yang telah dilaporkan selesai (*Earned Value*) dengan persentase target rencana awal proyek (*Planned Value*). Secara matematis, SPI dirumuskan pada Persamaan 1 berikut:

$$SPI = \frac{Earned\ Value\ (EV)}{Planned\ Value\ (PV)}$$

Keterangan Variabel:

a. SPI (*Schedule Performance Index*) : Nilai indeks efisiensi waktu pelaksanaan proyek.
b. EV (*Earned Value* / Nilai Hasil) : Persentase bobot pekerjaan aktual yang secara faktual telah diselesaikan di lapangan. Dalam sistem ini, nilai EV diakumulasikan secara otomatis dari progres tugas yang diinputkan oleh pekerja melalui formulir *daily report*.
c. PV (*Planned Value* / Nilai Rencana) : Persentase bobot target pekerjaan yang seharusnya telah diselesaikan pada titik waktu pemantauan tertentu berdasarkan jadwal rencana awal (*baseline*).

Di dalam arsitektur sistem ini, sumber data aktual diekstraksi secara berkelanjutan dari input *daily report* para pelaksana lapangan. Kecerdasan sistem (*smart system capability*) bekerja di latar belakang dengan mengakumulasi data mentah dari laporan harian tersebut dan secara otomatis mengkalkulasi rasio SPI tanpa memerlukan intervensi manual dari pihak administrator.

Otomatisasi ini kemudian terintegrasi langsung dengan antarmuka *dashboard monitoring*. Apabila hasil kalkulasi sistem memenuhi parameter kondisi seperti pada Persamaan 2 berikut:

$$SPI < 1$$

Kondisi tersebut mengindikasikan bahwa progres aktual tertinggal dari target rencana. Ketika parameter ini terpenuhi, EWS akan seketika memicu indikator peringatan visual—seperti perubahan warna status menjadi merah atau *flagging*

kondisi "Kritis"—pada layar *dashboard*. Melalui pendekatan pengawasan proaktif ini, sistem mentransformasikan tumpukan data rekapitulasi *daily report* menjadi wawasan visual yang instan, membebaskan manajer proyek dari beban perhitungan manual, serta memastikan bahwa evaluasi kinerja operasional dapat berjalan secara presisi, terpusat, dan sangat responsif (Umana dkk., 2022).

Melalui hasil komparasi matematis tersebut, sistem *dashboard* dirancang untuk menerjemahkan nilai numerik SPI menjadi status visibilitas proyek dengan parameter sebagai berikut:

a. $\boldsymbol{SPI = 1}$: Proyek berjalan tepat waktu sesuai dengan target jadwal (*On Schedule*). Indikator visual umumnya menampilkan status normal atau warna hijau.
b. $\boldsymbol{SPI > 1}$: Proyek berjalan lebih cepat dari target jadwal yang direncanakan (*Ahead of Schedule*).
c. $\boldsymbol{SPI < 1}$: Proyek mengalami keterlambatan operasional (*Behind Schedule*). Kondisi matematis inilah yang bertindak sebagai pemicu utama (*trigger*) bagi *Early Warning System* untuk secara otomatis menampilkan notifikasi kritis atau merubah indikator visual *dashboard* menjadi peringatan warna merah.

2.2.6 *Framework* Next.js

Next.js merupakan kerangka kerja (*framework*) front-end berbasis React yang dirancang untuk membangun antarmuka web berskala *enterprise* dengan performa tinggi. Keunggulan utamanya terletak pada dukungan *Server-Side Rendering* (SSR) dan arsitektur berbasis komponen (*component-based*), yang sangat ideal untuk memvisualisasikan elemen *dashboard* secara modular, seperti grafik progres kerja dan indikator warna *Red-Amber-Green* (RAG). Dalam implementasi *Early Warning System* (EWS) pada manajemen proyek, Next.js berperan krusial melalui mekanisme penarikan data asinkron. *Framework* ini mampu mengonsumsi akumulasi data *daily report* dan hasil kalkulasi *Schedule Performance Index* (SPI) dari *backend* secara *real-time*. Ketika terdeteksi perubahan status proyek menjadi kritis (misalnya $SPI < 1$),

antarmuka *dashboard* dapat merefleksikan peringatan visual tersebut secara instan tanpa perlu memuat ulang (*refresh*) keseluruhan halaman, sehingga manajer proyek dapat memperoleh informasi operasional yang faktual secara cepat dan interaktif (Genne, 2025).

2.2.7 *Unified Modeling Language*

UML (*Unified Modeling Language*) adalah bahasa pemodelan standar yang digunakan untuk memvisualisasikan, merancang, dan mendokumentasikan sistem perangkat lunak yang berorientasi objek. UML sangat berguna dalam menggambarkan kebutuhan pengguna dan alur sistem melalui berbagai diagram dan tabel data. UML menggunakan konsep desain berorientasi objek dan tidak bergantung pada bahasa pemrograman tertentu. Ini memungkinkan pemodelan sistem yang kompleks dan besar dengan cara yang terstruktur (Maggi dkk., 2020). UML mencakup berbagai jenis diagram yang dibagi menjadi diagram struktural dan diagram perilaku. Diagram kelas adalah salah satu yang paling umum digunakan untuk memodelkan tampilan desain statis dari system (Al-Fedaghi, 2021). Terdapat beberapa jenis diagram pada uml diantaranya yaitu, *Use Case Diagram*, *Sequence Diagram*, *Class Diagram*, dan *Activity Diagram*.

1) *Use Case Diagram*

Use case adalah abstraksi dari interaksi antara sistem dan aktor. *Use case* bekerja dengan cara mendeskripsikan tipe interaksi antara *user* sebuah sistem dengan sistemnya sendiri melalui sebuah cerita bagaimana sebuah sistem dipakai. *Use case* merupakan konstruksi untuk mendeskripsikan bagaimana sistem akan terlihat di mata user. Sedangkan *use case diagram* memfasilitasi komunikasi diantara analis dan pengguna serta antara analis dan klien (Ahmad, 2022).

Tabel 2.2 Simbol Use Case Diagram

Simbol	Nama	Keterangan
	Aktor	Mewakili peran orang atau pengguna, sistem yang lain dan alat ketika berkomunikasi dengan *use case*
	Use case	Abstraksi dan interaksi sistem dan aktor
	Association	Abstraksi dari penghubung antara aktor dan *use case*
	Generalisasi	Menunjukan spealisasi aktor untuk dapat berpartisipasi dengan *use case*
<<include>>	*Include*	Menunjukan bahwa suatu use case seluruhnya merupakan fungsional dari *use case* lain
<<extend>>	*Extend*	Menunjukan bahwa suatu *use case* merupakan tambahan fungsionalitas dari *use case* lainnya jika suatu kondisi terpenuhi

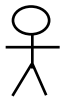
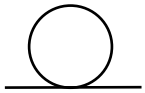
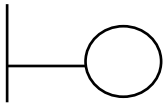
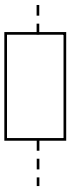
Sumber: (Suriya & Nivetha, 2023)

2) *Sequence Diagram*

Sequence diagram menunjukkan bagaimana sekelompok objek dapat berkolaborasi dalam beberapa behavior atau kebiasaan. *Sequence diagram* menggambarkan perilaku pada sebuah skenario. Kegunaan dari *sequence diagram*

adalah menunjukkan serangkaian proses yang dikirim antara objek yang berinteraksi dengan objek, sesuatu terjadi pada titik tertentu pada saat eksekusi sistem.

Tabel 2.3 Simbol Sequence Diagram

Simbol	**Nama**	**Keterangan**
	Aktor	Menggambarkan orang yang sedang berinteraksi dengan sistem.
	*Entity Class*	Menggambarkan hubungan yang akan dilakukan.
	*Boundary Class*	Menggambarkan sebuah gambaran dari *form*
	Control Class	Mengambarkan penghubung antara *boundary* dengan tabel
	*A focus of control & A life line*	Mengambarkan tempat mulai dan berakhirnya *message*
	A message	Menggambarkan pengiriman pesan

Sumber: (Suriya & Nivetha, 2023)

3) *Class Diagram*

Class adalah dekripsi kelompok obyek-obyek dengan properti, perilaku (operasi) dan relasi yang sama. Sehingga dengan adanya *class diagram* dapat memberikan pandangan global atas sebuah sistem. Hal tersebut tercermin dari *class-class* yang ada dan relasinya satu dengan yang lainnya. Sebuah sistem biasanya mempunyai beberapa *class diagram*. *Class diagram* sangat membantu dalam visualisasi struktur kelas dari suatu sistem.

Tabel 2.4 Simbol Class Diagram

Simbol	Nama	Keterangan
	Generalization	Hubungan dimana objek anak berbagi perilaku dan struktur data dari objek yang ada di atasnya objek induk
	Nary Association	Upaya untuk menghidari asosiasi dengan lebih dari 2 objek
	Class	Himpunan dari objek-objek yang berbagi atribut serta operasi yang sama
	Collaboration	Deskripsi dari urutan aksi-aksi yang menghasilkan suatu hasil yang terukur bagi suatu aktor
	Realization	Operasi yang benar-benar dilakukan oleh suatu objek
	Dependency	Hubungan dimana perubahan yang terjadi pada suatu elemen mandiri akan mempengaruhi elemen yang

Simbol	Nama	Keterangan
		bergantung padanya elemen yang tidak mandiri
	Association	Apa yang menghubungkan antara objek satu dengan objek lainnya

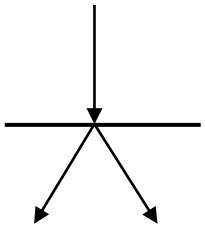
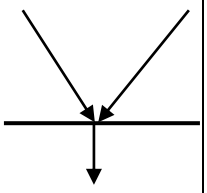
Sumber: (Suriya & Nivetha, 2023)

4) *Activity Diagram*

Activity diagram merupakan gambaran dari rangkaian aliran dari aktivitas. Tujuan dari *activity diagram* adalah mendeskripsikan aktivitas yang dibentuk dalam suatu operasi sehingga dapat digunakan aktivitas lain.

Tabel 2.5 Simbol Activity Diagram

Simbol	Nama	Keterangan
	Start point	Diletakkan pada pojok kiri atas penanda awal aktivitas
	End point	Akhir aktivitas
	Activities	Menggambar kan suatu proses atau kegiatan bisnis

Simbol	Nama	Keterangan
	*Fork*/percabangan	Digunakan untuk menunjukan kegiatan yang dilakukan secara pararel atau untuk mengabungkan dua kegiatan yang pararel menjadi satu
	*Join*/penggabungan	Digunakan untuk menunjukan adanya dekomposisi

Sumber: (Khomokhoana dkk., 2025)

2.2.8 *Entity Relationship Diagram*

Entity Relationship Diagram (ERD) merupakan alat pemodelan konseptual tahap awal untuk merepresentasikan entitas, atribut, dan hubungan dalam basis data secara grafis. ERD bermanfaat untuk memvisualisasikan struktur data sebelum tahap implementasi teknis dilakukan (Pulungan dkk., 2023), meminimalisir berbagai kesalahan desain, serta menjadi alat komunikasi visual yang efektif antara pengembang dan pemangku kepentingan. Secara struktural, komponen utama ERD terdiri dari entitas (objek data unik), atribut yang mendeskripsikan properti entitas beserta kunci primernya (’Afiifah dkk., 2022), dan hubungan yang menggambarkan interaksi antar entitas dalam sistem.

Tabel 2.6 Entity Relationship Diagram

No.	Simbol	Keterangan

1.		**Entitas**, yaitu representasi objek data diskrit yang memiliki identitas unik di dalam sistem.
2.		**Relasi**, yaitu keterkaitan hubungan yang mengikat antar-entitas dalam basis data. Relasi ini dapat diklasifikasikan ke dalam pemetaan one-to-one (satu ke satu), one-to-many (satu ke banyak), atau many-to-many (banyak ke banyak).
3.		**Atribut**, yaitu properti atau karakteristik spesifik yang menjabarkan informasi mendetail dari sebuah entitas maupun relasi.
4.		**Garis**, merupakan penghubung visual yang merangkai entitas dengan atributnya, serta menyambungkan himpunan entitas dengan relasinya.
5.		**Input/output**, yaitu indikator visual yang merepresentasikan alur keluar-masuknya data, parameter operasional, maupun informasi di dalam sistem.

Sumber: (Pulungan dkk., 2023)

2.2.9 Basis Data

Data base atau basis data merupakan kumpulan data yang memiliki format dan struktur-struktur tertentu sehingga memungkinkan sistem berbasis komputer dapat melakukan penyimpanan, pengelolaan, dan pengambilan data secara cepat. Dalam basis data juga terdapat sebuah sistem basis data, terdiri dari data secara logis saling terkait dan biasanya disimpan dalam sebuah repositori data. Karena pada sistem basis data penyimpanan data bersifat satu tempat, maka memungkinkan dalam saat bersamaan beberapa pengguna mengakses dan memperbarui basis data tersebut. Tempat untuk mengakses dan mengolah basis data ini dikenal sebagai DBMS (*Basis*

data Management System). Sebuah sistem basis data sendiri terdiri dari 5 komponen yaitu: perangkat keras, perangkat lunak, data, prosedur, dan manusia (Hadiprakoso, 2021).

1) Perangkat Keras

   DBMS membutuhkan *hardware* (perangkat keras) untuk dapat diinstal. Perangkat keras dapat berupa sebuah komputer pribadi sampai jaringan komputer tergantung pada kebutuhan organisasi dan DBMS yang digunakan. Beberapa DBMS hanya berjalan pada perangkat keras atau sistem operasi tertentu, sementara yang lain dapat dijalankan pada berbagai macam perangkat keras dan sistem operasi. Lebih lanjut lagi perangkat keras di sini dapat mengacu pada semua perangkat fisik sistem, termasuk komputer.

2) Perangkat Lunak

   Dalam DBMS terdapat 3 jenis perangkat lunak yaitu sistem operasi, DBMS, dan program aplikasi lainnya. Sistem operasi mengelola semua komponen perangkat keras dan memungkinkan semua perangkat lunak lain untuk diinstal dan berjalan di komputer, contoh perangkat lunak sistem operasi termasuk Microsoft, Linux, dan Mac OS. Selanjutnya DBMS sebagai pengelola basis data seperti, Microsoft SQL Server, Oracle, MySQL. Program aplikasi lainnya digunakan untuk mengakses data yang terdapat didalam basis data untuk menghasilkan laporan dan informasi lainnya.

3) Data

   Komponen yang paling penting dari sistem basis data dari sudut pandang pengguna adalah data. Data di sini meliputi pengumpulan fakta yang tersimpan dalam basis data. Karena data adalah bahan baku dari mana informasi yang dihasilkan, menentukan data yang masuk ke dalam basis data dan bagaimana mengatur data menjadi informasi yang berharga merupakan bagian penting dari sebuah sistem basis data.

4) Prosedur

   Prosedur merupakan sebuah aturan yang mengatur desain dan penggunaan sistem basis data.

5) Manusia

   Komponen ini mencakup semua pengguna dari sistem basis data. Berdasarkan fungsi pekerjaan utama masing-masing terdapat empat jenis pengguna yang dapat diidentifikasi dalam sistem basis data yakni: administrator basis data, perancang basis data, analis sistem, *programmer*, dan pengguna akhir.

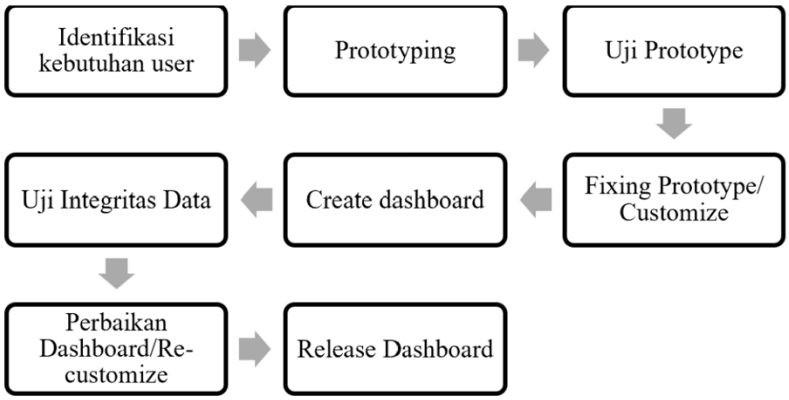
2.2.10 *Structure Query Language*

Structured Query Language (SQL) merupakan bahasa pemrograman standar yang dirancang secara spesifik untuk mengelola, memanipulasi, dan mengekstraksi informasi dari Sistem Manajemen Basis Data Relasional (RDBMS). Secara fungsional, SQL memfasilitasi interaksi tingkat tinggi antara aplikasi perangkat lunak dan repositori data melalui eksekusi kueri (*query*) terstruktur. Instruksi dalam SQL umumnya diklasifikasikan ke dalam beberapa kategori utama, di antaranya *Data Definition Language* (DDL) untuk merancang skema atau struktur tabel, serta *Data Manipulation Language* (DML) seperti perintah `SELECT`, `INSERT`, `UPDATE`, dan `DELETE` yang bertugas memproses data operasional. Keunggulan utama SQL terletak pada kemampuannya mengeksekusi fungsi agregasi komputasional, memfilter informasi spesifik, serta mengonsolidasikan entitas dari berbagai tabel yang saling berelasi melalui operasi *JOIN*, sehingga memungkinkan pengambilan maupun pengolahan data berskala besar secara cepat, akurat, dan terpusat di tingkat *database* (Khan dkk., 2023).

BAB III
METODE PENELITIAN

3.1 Metode Penelitian

Penelitian ini menggunakan metode pengembangan *prototyping* yang merupakan salah satu pendekatan dalam *System Development Life Cycle* (SDLC). Metode ini dipilih karena sesuai dengan tujuan penelitian untuk mengembangkan fitur dashboard pada project management system yang sudah berjalan di PT Smart Home Inovasi Yogyakarta, di mana diperlukan pemahaman kebutuhan pengguna secara mendalam serta umpan balik berulang untuk menyempurnakan antarmuka dan fungsionalitas dashboard berbasis data *daily report*. Pendekatan *prototyping* memungkinkan pengembang untuk membuat model awal sistem (*prototype*) yang kemudian dievaluasi langsung oleh pengguna—dalam hal ini Manajer Proyek dan Teknisi Lapangan—sehingga perbaikan dapat dilakukan secara bertahap hingga diperoleh fitur dashboard yang benar-benar sesuai dengan kebutuhan manajemen proyek.



Gambar 3.1 Metode Penelitian

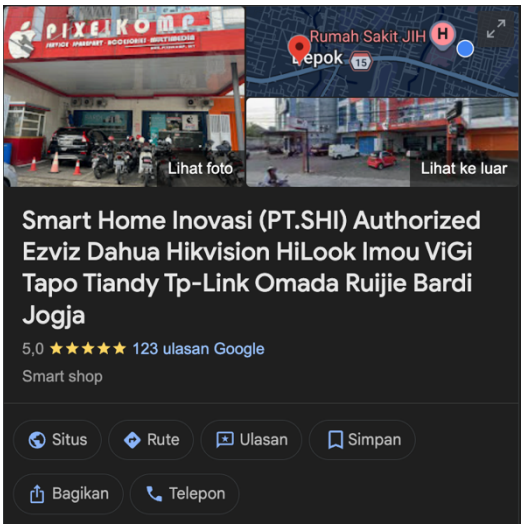
Adapun tahapan dalam metode *prototyping* meliputi:

1. **Identifikasi Kebutuhan *User*:** Menganalisis kebutuhan manajer dan admin PT SHI untuk mengubah tumpukan *daily report* yang berupa teks menjadi informasi visual, seperti status kesehatan proyek dan target vs aktual.
2. ***Prototyping*:** Membuat rancangan desain awal (*mockup*) antarmuka *dashboard*, termasuk tata letak grafik, *progress bar*, dan indikator warna *Red-Amber-Green* (RAG).
3. **Uji *Prototype*:** Mempresentasikan desain awal tersebut kepada pengguna (manajer PT SHI) untuk dinilai apakah tampilannya sudah intuitif dan mempermudah pemantauan.
4. ***Fixing Prototype / Customize*:** Merevisi dan menyesuaikan desain *dashboard* (tata letak, jenis grafik, warna) berdasarkan masukan dari manajer pada tahap pengujian awal.
5. ***Create Dashboard*:** Membangun antarmuka *dashboard* secara nyata menggunakan *framework* Next.js. Pada tahap ini, komponen visual dikembangkan dan logika otomatisasi perhitungan (SPI) ditanamkan agar sistem dapat merender ekstraksi data *daily report* secara cepat dan asinkronus.
6. **Uji Integritas Data:** Menguji sistem menggunakan data laporan harian asli milik PT SHI untuk memastikan perhitungan agregasi kemajuan proyek dan perubahan warna peringatan dini (EWS) berjalan akurat.
7. **Perbaikan *Dashboard / Re-customize*:** Memperbaiki *bug* pada baris kode jika masih terdapat kesalahan dalam kalkulasi agregasi data atau tampilan visual saat pengujian.
8. ***Release Dashboard*:** Meluncurkan fitur *dashboard* pemantauan yang sudah stabil ke dalam aplikasi manajemen proyek PT SHI agar siap digunakan secara operasional.

3.2 Obyek Penelitian

3.2.1 Gambaran Umum

PT Smart Home Inovasi Yogyakarta (SHI) merupakan sebuah perusahaan teknologi yang berlokasi di Kabupaten Sleman, Daerah Istimewa Yogyakarta. Perusahaan ini mengkhususkan diri pada pengembangan dan implementasi solusi *Smart Home* serta *Internet of Things* (IoT). Mengusung visi sebagai penyedia solusi hunian cerdas terdepan, PT SHI menghadirkan berbagai inovasi otomasi perangkat—mulai dari sistem keamanan, pengaturan suhu, hingga efisiensi energi—guna menciptakan ekosistem hunian yang aman, nyaman, dan hemat energi bagi kliennya.



Gambar 3.2 Gambaran Umum

3.2.2 Proses Bisnis dan Aktor yang Terlibat

Dalam sistem manajemen proyek yang diusulkan pada penelitian ini, terdapat desentralisasi wewenang pelaporan untuk memangkas birokrasi administratif. Adapun aktor utama yang terlibat dan hak aksesnya dalam proses bisnis pencatatan *daily report* adalah sebagai berikut:

1. Teknisi Lapangan: Aktor yang bertugas sebagai pelaksana operasional di lokasi klien (misalnya melakukan instalasi IoT). Dalam sistem ini, teknisi diberikan hak akses untuk menginputkan data *daily report* (berupa persentase progres pekerjaan

harian) secara langsung ke dalam sistem, tanpa harus melaporkannya terlebih dahulu secara manual atau tekstual melalui perantara.

2. Manajer Proyek: Aktor pada tingkat manajerial yang bertanggung jawab mengawasi seluruh portofolio proyek. Manajer bertindak sebagai pengguna utama antarmuka *dashboard*. Aktor ini tidak lagi bertugas menginput data laporan dari teknisi, melainkan fokus pada mengevaluasi *Project Health Status*, memantau grafik kemajuan proyek, dan merespons peringatan dini (*Early Warning System*) yang dihasilkan oleh sistem.

Alur Proses Bisnis Pemantauan Proyek (Setelah Pengembangan):

Proses bisnis pemantauan proyek melalui *dashboard* ini dirancang untuk bekerja secara otomatis dan *real-time* dengan tahapan operasional sebagai berikut:

1. Pencatatan Progres Aktual: Setelah menyelesaikan pekerjaan harian di lokasi klien, Teknisi Lapangan membuka aplikasi dan langsung mengisi *form daily report* yang mencakup rincian tugas dan persentase penyelesaian (sebagai *Earned Value*).
2. Kalkulasi Data Asinkronus: Data yang disubmit oleh teknisi secara otomatis masuk ke dalam *database*. Di latar belakang, sistem (*backend*) menghitung rasio komparasi *Schedule Performance Index* (SPI) dengan membandingkan progres aktual teknisi terhadap target rencana jadwal (*Planned Value*).
3. Visualisasi *Dashboard* dan EWS: Hasil kalkulasi dari sistem langsung dirender secara visual pada *dashboard* manajer menggunakan *framework* Next.js. Data tekstual diubah menjadi grafik progres dan metrik warna status (EWS).
4. Evaluasi Manajerial: Manajer Proyek membuka *dashboard* dan langsung dapat melihat proyek mana yang berstatus aman (hijau) dan mana yang mengalami keterlambatan atau kritis (merah) secara sekilas, memungkinkan pengambilan tindakan penanganan sedini mungkin.

BAB IV
ANALISIS DAN PERANCANGAN

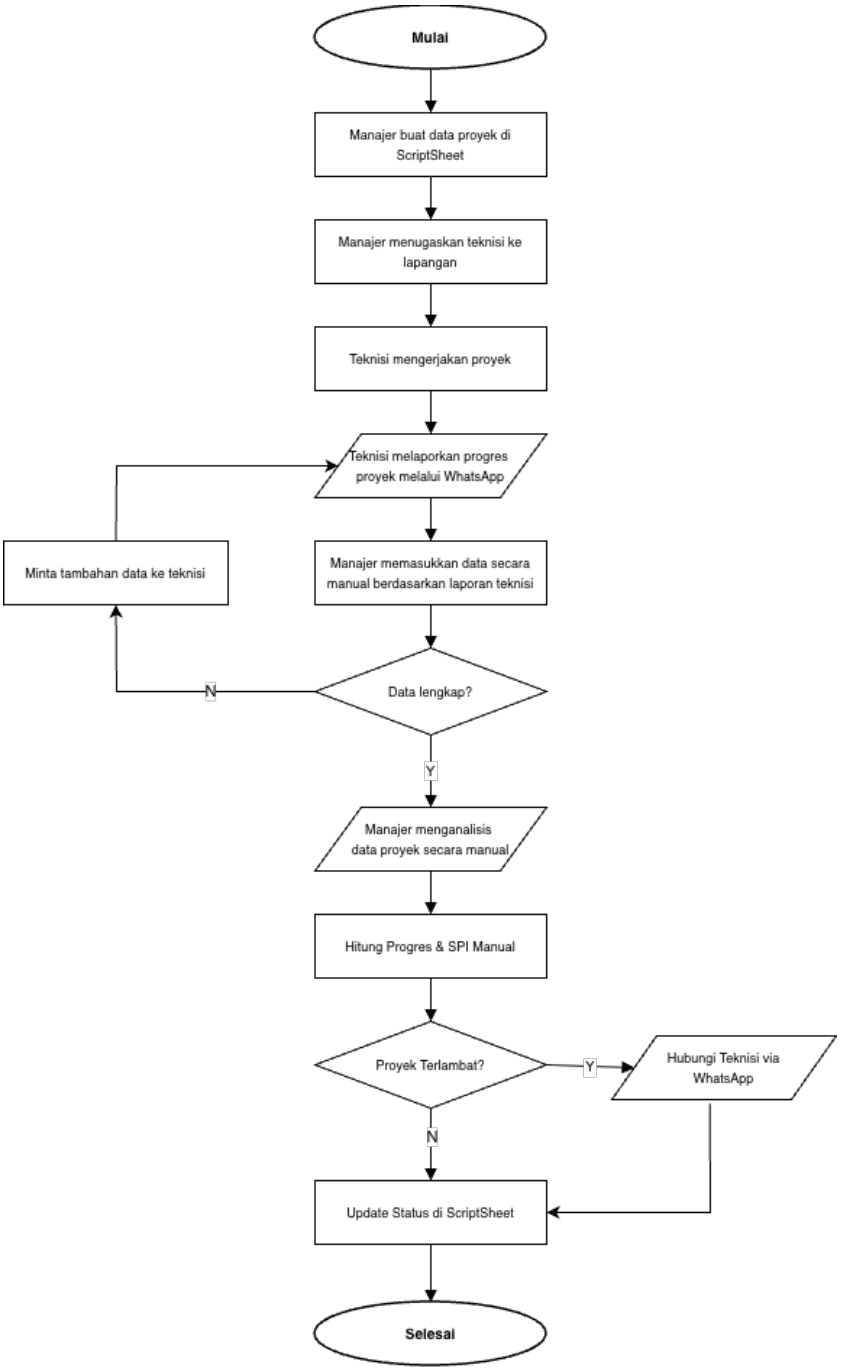
4.1 Analisis Sistem

Analisis Sistem adalah suatu teknik atau metode pemecahan masalah dengan cara menguraikan sistem ke dalam komponen-komponen pembentuknya untuk mengetahui bagaimana komponen-komponen tersebut bekerja dan saling berinteraksi satu sama lain untuk mencapai tujuan sistem. Analisis sistem biasanya dilakukan dalam membuat *System Design*. *System Design* adalah salah satu langkah dalam teknik pemecahan masalah di mana komponen-komponen pembentuk sistem digabungkan sehingga membentuk satu kesatuan sistem yang utuh. Hasil dari *System Design* merupakan gambaran sistem yang sudah diperbaiki. Teknik dari *System Design* ini meliputi proses penambahan, penghilangan, dan pengubahan komponen-komponen dari sistem semula.

4.1.1 Analisa Sistem yang Berjalan

Berdasarkan pengamatan terhadap proses bisnis di PT Smart Home Inovasi Yogyakarta, sistem pemantauan proyek saat ini masih bersifat manual dan terdesentralisasi. Alur operasional pada gambar 4.1 dimulai ketika Manajer Proyek menyusun data penugasan melalui aplikasi *spreadsheet* (ScriptSheet). Setelah teknisi menyelesaikan pekerjaan di lapangan, proses pelaporan progres harian beserta bukti dokumentasi hanya dikirimkan melalui aplikasi WhatsApp. Manajer harus memindahkan data pelaporan tersebut secara manual dari WhatsApp ke dalam rekapitulasi *spreadsheet*. Lebih lanjut, proses evaluasi kinerja jadwal berjalan dengan cara manajer menganalisis persentase progres proyek secara manual untuk mendeteksi status keterlambatan masing-masing proyek. Sistem yang konvensional ini sangat rentan terhadap *human error* dan menciptakan *information lag* (jeda waktu informasi) yang signifikan, sehingga tindakan mitigasi proyek menjadi lambat akibat waktu

manajer yang terkuras habis untuk mengurus administrasi data alih-alih melakukan pemantauan proaktif.

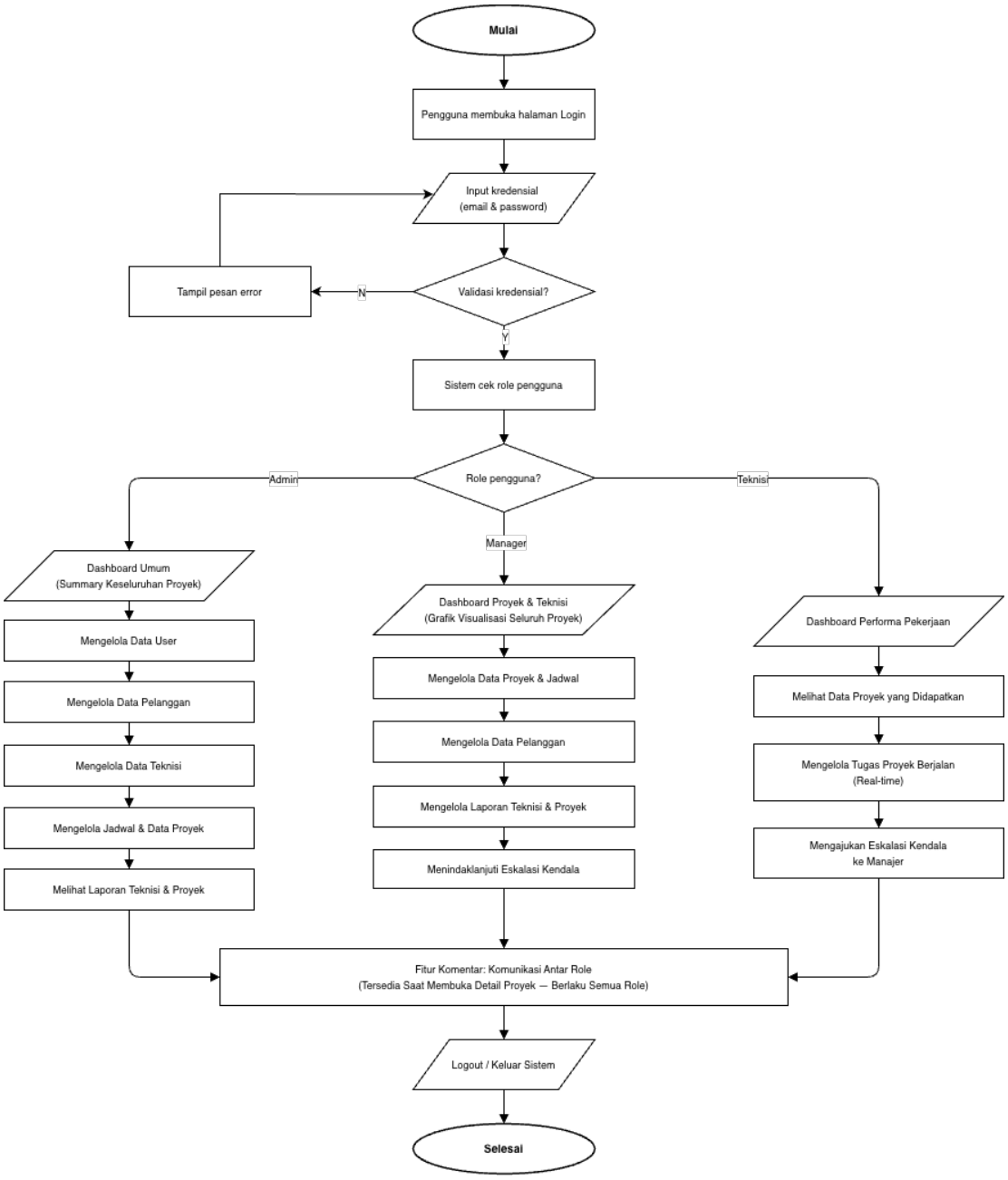


Gambar 4.1 Bagan Alur Sistem yang Berjalan

4.1.2 Analisa Sistem yang Diusulkan

Pada sistem operasional yang berjalan sebelumnya, manajemen menghadapi keterbatasan visibilitas dalam melacak metrik performa masing-masing teknisi, serta proses evaluasi yang bergantung pada metode manual sehingga mengakibatkan deteksi kendala sering kali terlambat (*information lag*). Keterlambatan mitigasi ini sering menimbulkan efek domino yang mengganggu linimasa proyek selanjutnya dan berpotensi menurunkan kepercayaan pelanggan. Oleh karena itu, implementasi *dashboard* pemantauan diusulkan sebagai solusi strategis untuk mentransformasikan manajemen proyek menjadi lebih efektif. Sistem yang mengintegrasikan otomasi kalkulasi kinerja ini akan meningkatkan otomatisasi alur proses bisnis, meminimalisasi risiko *human error* dalam rekapitulasi, serta memastikan manajer dapat melakukan pengawasan dan evaluasi secara *real-time* demi menjaga mutu penyelesaian proyek.

Untuk mewujudkan solusi tersebut, alur sistem usulan pada gambar 4.2 dirancang secara terpusat yang dimulai dari proses validasi *login* untuk mendeteksi hak akses (*role*) pengguna. Setelah autentikasi berhasil, sistem akan mengarahkan pengguna ke ruang kerja spesifik; Admin diarahkan ke *Dashboard* Umum untuk mengelola data *master* pengguna dan keseluruhan data proyek, Manajer Proyek diarahkan ke *Dashboard* pemantauan untuk mengawasi visualisasi grafis operasional dan menindaklanjuti eskalasi, sedangkan Teknisi mengakses *Dashboard* Performa untuk melaporkan progres tugas harian yang dikunci otomatis secara *real-time* serta untuk melaporkan kendala lapangan. Seluruh aktivitas dari ketiga *role* ini kemudian didukung oleh fitur komentar global pada detail proyek untuk komunikasi terpadu, dan siklus proses bisnis ini akan berakhir ketika pengguna melakukan *logout* dari sistem.



Gambar 4.2 Bagan Alur Sistem yang Diusulkan

4.2 Analisa Kebutuhan

Pada tahap ini, analisis dilakukan untuk mengidentifikasi kebutuhan spesifik dari PT Smart Home Inovasi Yogyakarta terkait pengembangan fitur *dashboard*

pemantauan proyek berbasis data *daily report*. Kebutuhan sistem ini dibagi menjadi dua, yaitu analisa kebutuhan fungsional dan analisa kebutuhan non-fungsional.

4.2.1 Analisa Kebutuhan User

Berdasarkan desentralisasi wewenang pelaporan dan alur proses bisnis yang diusulkan pada PT Smart Home Inovasi Yogyakarta, terdapat dua kategori pengguna utama yang akan berinteraksi langsung dengan sistem pemantauan ini, yaitu Teknisi Lapangan dan Manajer Proyek. Peran Admin tidak dimasukkan karena bertindak sebagai pengelola sistem (*system administrator*), bukan sebagai pengguna akhir (*end-user*) operasional yang menjalankan fungsi harian proyek. Adapun rincian kebutuhan dari masing-masing aktor adalah sebagai berikut:

1. Teknisi Lapangan

   Teknisi Lapangan berperan sebagai pelaksana operasional di lokasi klien, seperti melakukan instalasi perangkat IoT. Dalam sistem ini, teknisi bertindak sebagai sumber utama pencatatan data faktual. Kebutuhan Teknisi Lapangan meliputi:

   a. Teknisi dapat melakukan input *Daily Report* mandiri**.** Teknisi dapat mengisi langsung formulir laporan progres harian segera setelah menyelesaikan pengerjaan di lokasi klien, yang mencakup rincian tugas, persentase *Earned Value*, dan catatan kendala operasional tanpa melalui perantara manajer.
   b. Teknisi dapat mengakses *Self-Performance Dashboard*. Ini memungkinkan teknisi untuk melihat visualisasi ringkas nilai *Schedule Performance Index* (SPI) pribadi yang dikalkulasi secara otomatis dari akumulasi data laporan harian mereka, guna melakukan swamonitoring status kinerja jadwal secara mandiri (apakah sesuai target, mendahului, atau terlambat).

2. Manajer Proyek

   Manajer Proyek berfungsi sebagai pengawas pada tingkat manajerial yang bertanggung jawab atas kelancaran seluruh portofolio proyek perusahaan. Pada pengembangan sistem ini, Manajer Proyek tidak lagi bertugas menginput laporan secara manual, melainkan bertindak sebagai pengguna utama antarmuka visual. Kebutuhan Manajer Proyek meliputi:

a. Mengakses halaman *dashboard* utama yang memvisualisasikan ringkasan seluruh proyek aktif.
b. Memantau metrik *Project Health Status* yang direpresentasikan melalui indikator visual kode warna (*Red, Amber, Green*) untuk setiap proyek.
c. Melihat perbandingan terperinci antara progres ideal (*Planned Value*) berdasarkan durasi proyek, dengan progres aktual (*Earned Value*) hasil laporan harian.
d. Mengidentifikasi nilai deviasi progres dan membaca catatan kendala teknis terakhir yang dihadapi teknisi.
e. Menerima peringatan dini (*Early Warning System*) melalui penonjolan urutan proyek berstatus kritis pada *dashboard*, sehingga dapat segera menentukan prioritas penanganan secara objektif.
f. Hak akses Manajer Proyek dibatasi pada pemantauan *dashboard* dan evaluasi kinerja operasional, tanpa wewenang atau kebutuhan untuk mengelola data di luar lingkup *daily report* (seperti data keuangan atau pengadaan material).

4.2.2 Analisa Kebutuhan Fungsional

Analisa kebutuhan fungsional mendefinisikan layanan, fitur, dan fungsi yang harus disediakan oleh sistem agar dapat berjalan sesuai dengan alur bisnis perusahaan. Kebutuhan fungsional pada sistem ini adalah sebagai berikut:

1. Sistem Penginputan Data Operasional

   Sistem harus memfasilitasi Teknisi Lapangan untuk melakukan *input* data *daily report* secara langsung, yang mencakup rincian persentase progres pekerjaan harian (*Earned Value*) dan catatan kendala di lapangan.
2. Kalkulasi Matematis Otomatis

   Sistem (*backend*) harus mampu mengakumulasi data mentah dari *daily report* dan melakukan kalkulasi nilai deviasi progres atau *Schedule Performance Index* (SPI) secara otomatis dengan membandingkan progres aktual terhadap target rencana jadwal (*Planned Value*) (Radman dkk., 2025).
3. Pemicu *Early Warning System* (EWS)

Sistem harus mampu mengkategorikan status kesehatan proyek (*Project Health Status*) secara otomatis ke dalam indikator kode warna *Red-Amber-Green* (RAG) berdasarkan parameter perhitungan SPI (misalnya, berstatus Merah/Kritis jika nilai $SPI < 1$).

4. Visualisasi *Dashboard* Sentral

   Sistem harus dapat menampilkan antarmuka *dashboard* bagi Manajer Proyek yang menyajikan ringkasan seluruh proyek aktif, mengurutkan proyek berdasarkan tingkat urgensi/kekritisan, serta menampilkan detail berupa progres ideal, progres aktual, deviasi, dan indikator EWS secara visual tanpa memerlukan penelusuran dokumen tekstual.

4.2.3 Analisa Kebutuhan Non Fungsional

Analisa kebutuhan non-fungsional mendefinisikan batasan, standar performa, dan karakteristik operasional yang harus dimiliki oleh sistem. Kebutuhan non-fungsional pada pengembangan sistem ini meliputi:

1. Kinerja (*Performance*):

   Fitur *dashboard* harus mampu merender visualisasi data agregasi dan perubahan status *Early Warning System* secara asinkronus dan *real-time*. Penggunaan *framework* Next.js dengan fitur *Server-Side Rendering* (SSR) diwajibkan agar pembaruan informasi pada *dashboard* dapat berjalan cepat tanpa harus memuat ulang (*refresh*) keseluruhan halaman.

2. Keamanan (*Security*) dan Otorisasi:

   Sistem harus membedakan hak akses secara ketat (*role-based access control*). Teknisi Lapangan hanya memiliki wewenang untuk memasukkan data *daily report*, sedangkan hak akses untuk memantau indikator *dashboard* secara menyeluruh dikhususkan bagi Manajer Proyek.

3. Kegunaan (*Usability*):

Antarmuka *dashboard* harus dirancang dengan pendekatan visual yang intuitif, sehingga Manajer Proyek dapat mengidentifikasi kondisi kesehatan suatu proyek secara sekilas tanpa membutuhkan proses analitik manual.

4.3 Perancangan Sistem

Tahap perancangan sistem disusun untuk memberikan gambaran menyeluruh terkait model proses menggunakan *Unified Modeling Language* (UML), model data, struktur fisik basis data, relasi antar tabel, hingga perancangan antarmuka visual yang akan dikembangkan pada fitur *dashboard* pemantauan proyek ini. Proses perancangan dilakukan untuk memastikan bahwa seluruh komponen, mulai dari alur penginputan data *daily report* di lapangan hingga otomasi visualisasi *Early Warning System* (EWS), dapat saling terintegrasi dengan baik. Hal ini bertujuan agar sistem yang dibangun mampu beroperasi secara akurat dan memenuhi seluruh kebutuhan fungsional maupun non-fungsional dari Manajer Proyek dan Teknisi Lapangan di PT Smart Home Inovasi Yogyakarta. Perancangan sistem ini diuraikan secara lebih rinci ke dalam beberapa sub-bagian berikut.

4.3.1 Perancangan Model Proses

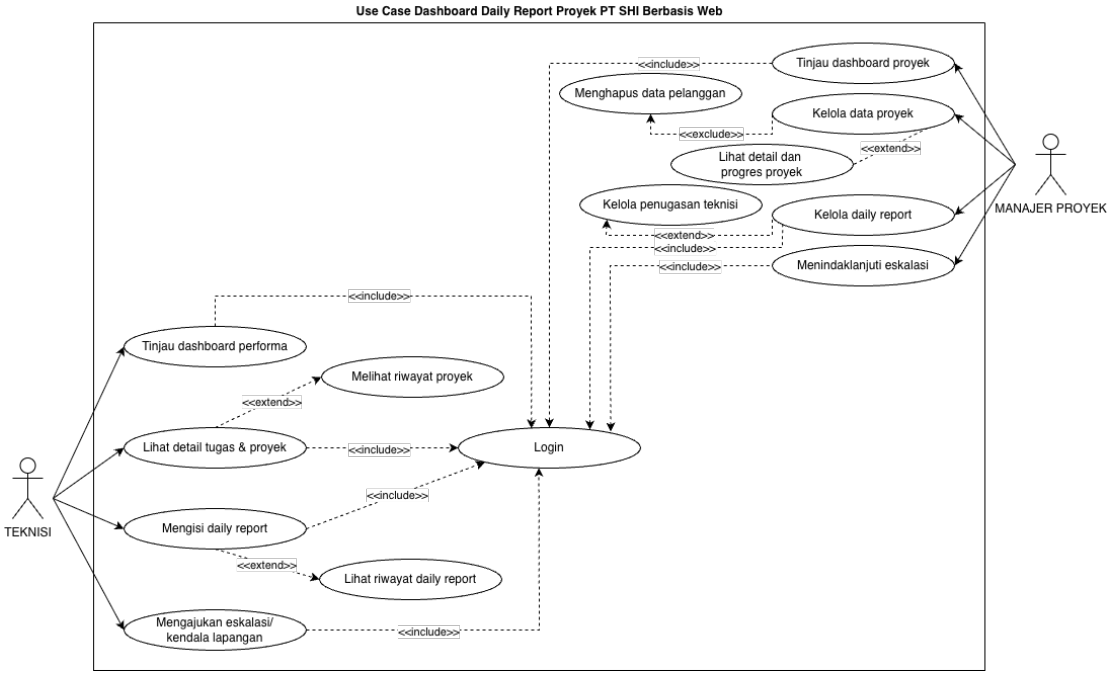
Perancangan model proses pada pengembangan sistem ini menggunakan *Unified Modeling Language* (UML) untuk memvisualisasikan interaksi antara pengguna (Teknisi Lapangan dan Manajer Proyek), sistem, dan basis data. Penggunaan UML bertujuan memetakan alur transformasi data *daily report* menjadi indikator visual pada *dashboard* secara terstruktur. Pendekatan ini dilakukan untuk meminimalisasi potensi kesalahan implementasi, mempermudah pemahaman logika *Early Warning System* (EWS), serta memastikan seluruh fungsionalitas terintegrasi secara efisien sesuai dengan kebutuhan operasional di PT Smart Home Inovasi Yogyakarta.

4.3.1.1 Model Use Case Diagram

Gambar 4.3 memodelkan interaksi fungsional antara dua aktor utama, yaitu Teknisi dan Manajer Proyek, terhadap sistem *Dashboard Daily Report* Proyek PT SHI. Sebagai fondasi keamanan, seluruh fungsionalitas inti dalam sistem ini diikat oleh

relasi <<*include*>> terhadap *use case* Login, yang berarti proses autentikasi wajib dilakukan sebelum pengguna mengakses sistem. Aktor Teknisi sebagai pelaksana lapangan memiliki wewenang untuk Tinjau dashboard performa, Lihat detail tugas & proyek, Mengisi *daily report*, dan Mengajukan eskalasi/kendala lapangan. Saat berinteraksi dengan *dashboard* performa dan form laporan harian, sistem memberikan opsi fungsionalitas tambahan secara dinamis (relasi <<*extend*>>) bagi Teknisi untuk masing-masing Melihat riwayat proyek dan Lihat riwayat daily report.

Di sisi lain, aktor Manajer Proyek memiliki hak manajerial untuk mengeksekusi fungsi Tinjau dashboard proyek, Kelola data proyek, Kelola daily report, serta Menindaklanjuti eskalasi. Saat melakukan pengelolaan data proyek, Manajer diberikan opsi fungsionalitas perluasan (<<*extend*>>) untuk Lihat detail dan progres proyek, namun sistem membatasi wewenang pengelolaan tersebut melalui relasi <<*exclude*>> yang secara mutlak mencegah tindakan Menghapus data pelanggan. Lebih lanjut, ketika Manajer sedang berinteraksi dengan fungsi laporan (Kelola *daily report*), sistem mengakomodasi opsi lanjutan (<<*extend*>>) agar Manajer dapat secara langsung mengambil tindakan operasional berupa Kelola penugasan teknisi sehingga bisa diukur sebagai parameter kesehatan proyek.



Gambar 4.3 Diagram Use Case

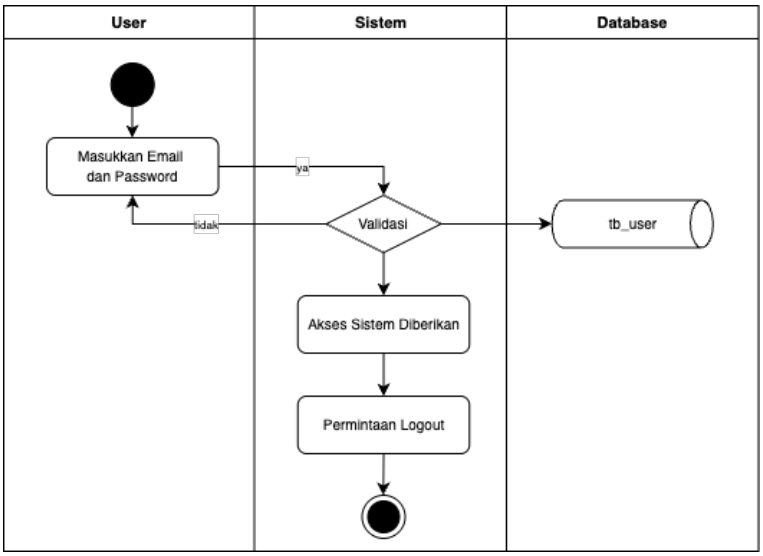
4.3.1.2 Model Activity Diagram

Subbab ini memaparkan *Activity Diagram* yang memvisualisasikan alur kerja dinamis dan rangkaian aktivitas operasional dari sistem *dashboard* yang diusulkan. Pemodelan ini berfungsi sebagai penjabaran langkah demi langkah dari *Use Case Diagram* sebelumnya, guna memperjelas bagaimana setiap proses bisnis dieksekusi di dalam sistem—mulai dari tahap autentikasi pengguna, pengisian laporan harian oleh teknisi, hingga proses pengawasan dan validasi oleh manajer proyek—beserta titik-titik pengambilan keputusan (*decision node*) yang terjadi di dalamnya.

1) *Activity Diagram* Autentikasi (*Login* dan *Logout)*

Gambar 4.4 mengilustrasikan proses autentikasi sistem, dimana proses dimulai ketika pengguna memasukkan data kredensial pada form login. Sistem kemudian melakukan validasi dengan mencocokkan data tersebut pada tabel *tb_user* di basis data. Jika data sesuai, sistem akan memberikan akses kepada pengguna untuk masuk ke dalam sistem. Namun, apabila data tidak valid, pengguna akan diminta untuk kembali mengisi ulang informasi yang benar. Setelah mendapatkan akses, pengguna dapat

menggunakan sistem hingga pada akhirnya mengirimkan permintaan *logout* sebagai tanda keluar dari sesi penggunaan.



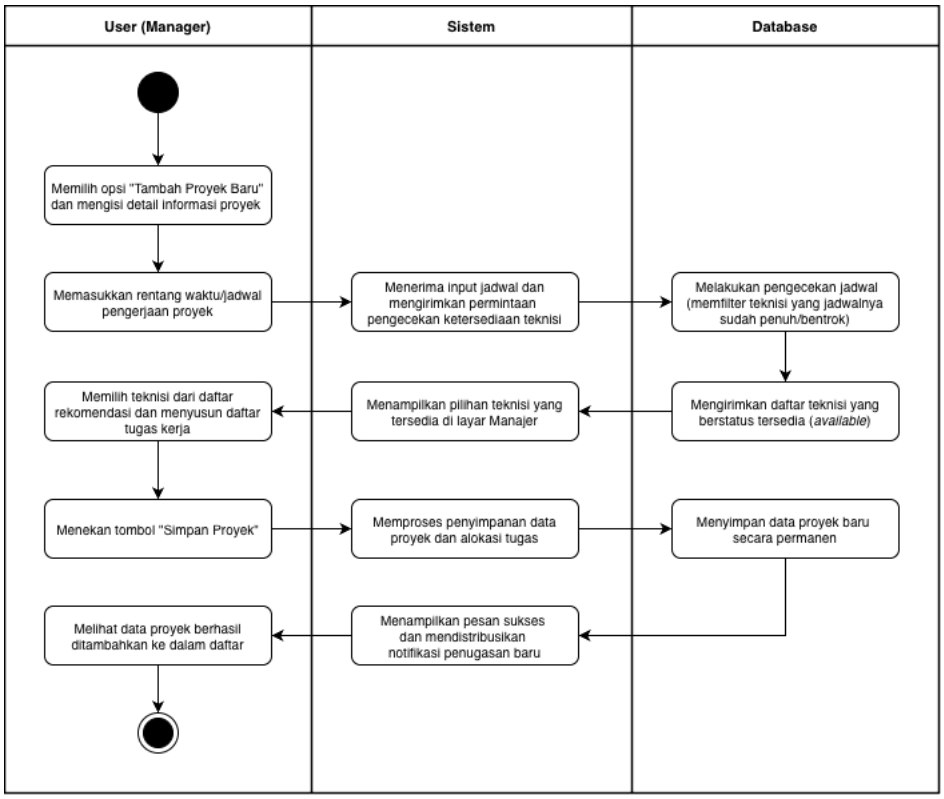
Gambar 4.4 Activity Diagram Autentikasi

2) *Activity Diagram* Pengelolaan Proyek

Gambar 4.5 merepresentasikan alur komputasi dan interaksi antarentitas dalam proses penambahan entitas proyek baru beserta mekanisme alokasi penugasannya. Alur operasional diinisiasi oleh aktor Manajer melalui antarmuka sistem dengan memilih opsi penambahan proyek, dilanjutkan dengan pengisian detail spesifikasi proyek dan penentuan rentang waktu kalender pengerjaannya. Input jadwal tersebut bertindak sebagai parameter kritis; sistem menangkap masukan tersebut dan meneruskannya sebagai kueri permintaan (*request*) kepada *database* untuk memvalidasi ketersediaan sumber daya manusia. Pada tingkat basis data, terjadi eksekusi pemrosesan penyaringan (*filtering*) guna menyeleksi dan mengeliminasi teknisi yang jadwal kerjanya berbenturan (*overlapping*). *Database* selanjutnya mengembalikan himpunan data teknisi yang secara eksklusif berstatus tersedia (*available*), yang kemudian dirender oleh sistem.

Berbekal data teknisi yang telah tervalidasi tersebut, Manajer melanjutkan proses tata kelola dengan menetapkan teknisi terkait dan merancang dekomposisi

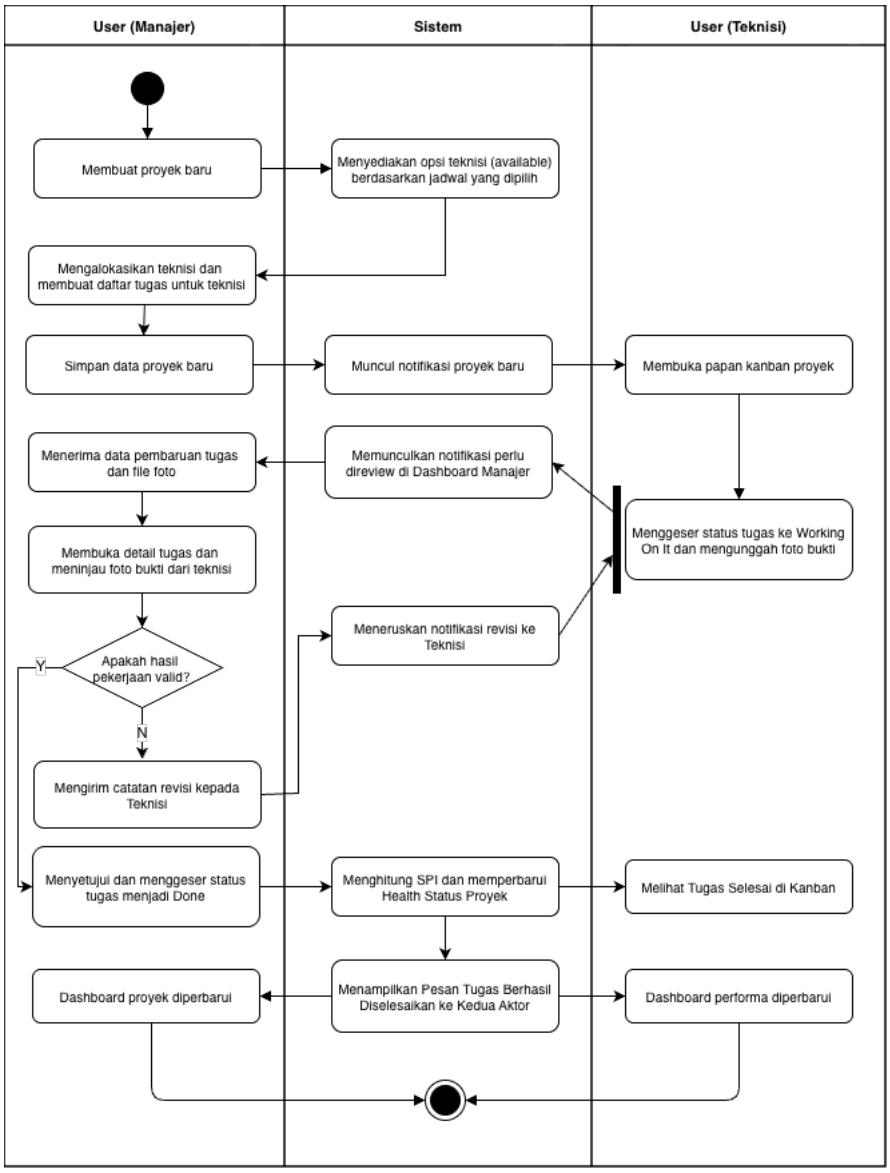
struktur kerja (*task breakdown*). Pemecahan rincian tugas ini memegang peranan krusial, mengingat setiap unit tugas (*task*) yang dialokasikan akan berfungsi sebagai parameter dasar dan tolok ukur komputasi dalam mengevaluasi status kesehatan proyek (*health status*) seiring dengan rasio penyelesaian tahapan kerja oleh teknisi. Rangkaian instruksi operasional ini dieksekusi secara sistematis saat Manajer mengonfirmasi penyimpanan proyek. Sistem merespons dengan memproses kompilasi data proyek beserta relasi penugasannya, untuk kemudian ditransmisikan ke *database* guna penyimpanan permanen (*persistent storage*). Sebagai bentuk umpan balik sistem (*system feedback*) dalam menutup siklus operasional, aplikasi secara otomatis mendistribusikan notifikasi penugasan baru dan merender pesan konfirmasi keberhasilan pada antarmuka Manajer. Siklus aktivitas ini mencapai titik akhir (*end state*) ketika entitas proyek baru tersebut telah terintegrasi secara utuh ke dalam hierarki daftar proyek pada sistem.



Gambar 4.5 Activity Diagram Pengelolaan Proyek

3) *Activity Diagram* Pelaporan Progres Harian, Validasi Tugas, & Kalkulasi SPI (*Review Gate*)

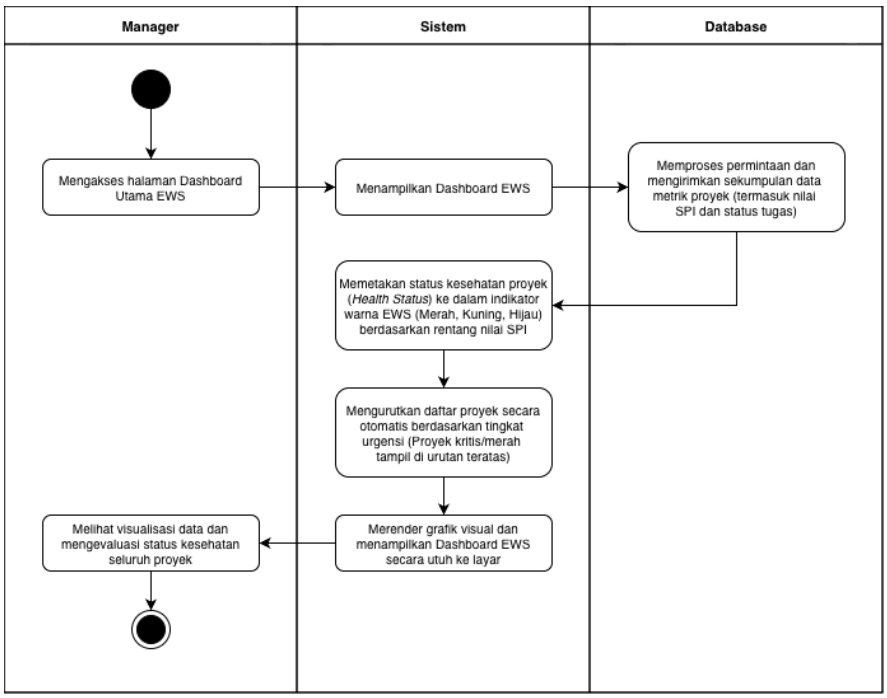
Gambar 4.6 ini mengilustrasikan siklus penugasan, pelaporan, dan validasi pekerjaan. Proses diawali oleh Manajer Proyek yang membuat data proyek berdasarkan jadwal tertentu. Sistem kemudian secara otomatis memfilter dan menampilkan rekomendasi teknisi yang tersedia pada jadwal tersebut untuk mencegah terjadinya tumpang tindih (*bentrok*) penugasan. Setelah Manajer mengalokasikan teknisi dan tugas, sistem akan mengirimkan notifikasi. Teknisi merespons penugasan melalui papan *kanban* dengan menggeser status tugas menjadi "*Working On It*" dan mengunggah foto bukti pekerjaan. Pelaporan ini menginisiasi fase validasi (*Review Gate*), di mana Manajer meninjau bukti tersebut. Jika belum sesuai, Manajer mengirimkan catatan revisi. Sebaliknya, jika valid, Manajer menyetujui tugas menjadi "*Done*". Persetujuan ini memicu sistem untuk mengkalkulasi ulang indikator kinerja (SPI) dan *Health Status* proyek, yang langsung memperbarui tampilan *dashboard* Manajer maupun Teknisi secara *real-time*.



Gambar 4.6 Activity Diagram Pelaporan Progres Harian, Validasi Tugas, & Kalkukasi SPI (Review Gate)

4) ***Activity Diagram Dashboard Early Warning System***

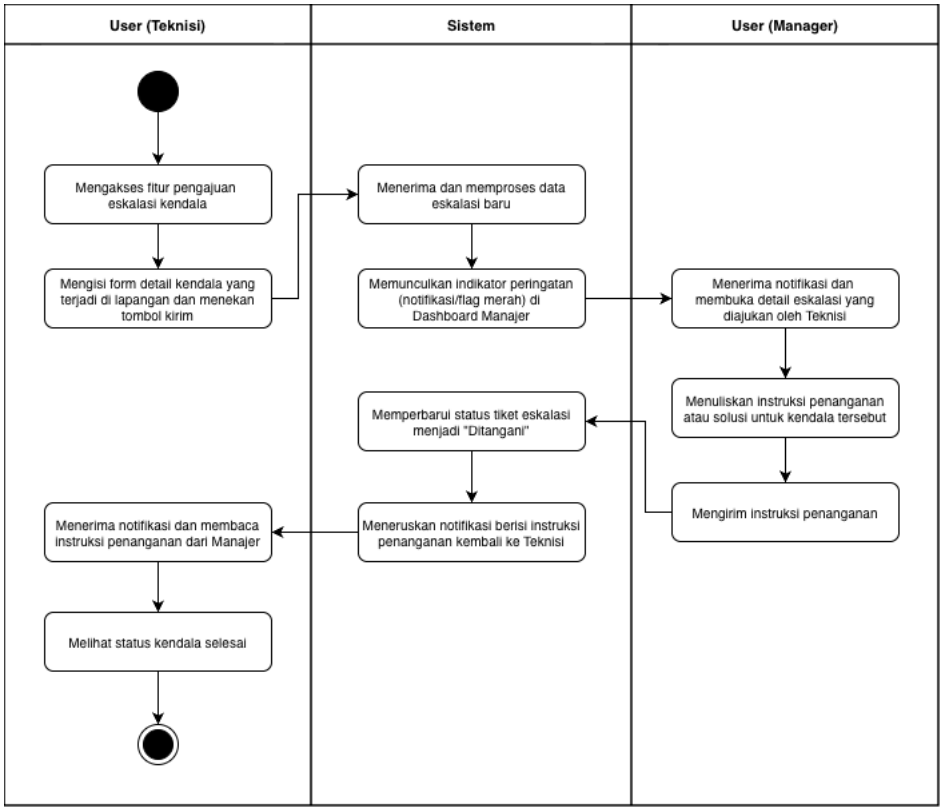
Gambar *4.7* memvisualisasikan alur sistematis dalam proses pemantauan *Dashboard Early Warning System* (EWS). Alur komputasi ini diinisiasi ketika Manajer mengakses halaman utama *dashboard*, yang kemudian direspons oleh sistem dengan meneruskan permintaan ekstraksi data ke *database*. Selanjutnya, *database* memproses permintaan tersebut dan mengembalikan sekumpulan data metrik proyek secara komprehensif, mencakup nilai *Schedule Performance Index* (SPI) beserta status tugas masing-masing proyek. Berdasarkan data yang diterima, sistem melakukan pemrosesan analitik lanjutan dengan memetakan status kesehatan proyek (*Health Status*) ke dalam indikator warna EWS—yakni merah, kuning, atau hijau—yang disesuaikan dengan parameter rentang nilai SPI. Kemudian sistem secara cerdas mengurutkan daftar proyek berdasarkan tingkat urgensi, sehingga proyek dengan status kritis (merah) secara otomatis ditempatkan pada urutan teratas. Alur ini berakhir ketika antarmuka *dashboard* berhasil ditampilkan secara utuh, sehingga Manajer dapat langsung mengevaluasi status kesehatan seluruh proyek.



Gambar 4.7 Activity Diagram Dashboard Early Warning System

5) *Activity Diagram* Pengajuan & Penanganan Eskalasi

Gambar *4.8* menjelaskan alur komunikasi dua arah dalam proses pengajuan dan penanganan eskalasi kendala. Proses diawali oleh Teknisi yang melaporkan detail kendala lapangan melalui form eskalasi. Laporan ini kemudian diproses oleh sistem untuk memunculkan indikator peringatan (*flag* merah) pada *dashboard* Manajer. Selanjutnya, Manajer meninjau detail masalah tersebut dan mengirimkan instruksi penanganan sebagai solusi. Tindakan ini memicu sistem untuk secara otomatis memperbarui status tiket eskalasi menjadi "Ditangani" dan meneruskan instruksi balasan tersebut kembali kepada Teknisi. Alur ini berakhir setelah Teknisi menerima notifikasi instruksi dari Manajer dan melihat bahwa status penanganan kendala tersebut telah selesai.



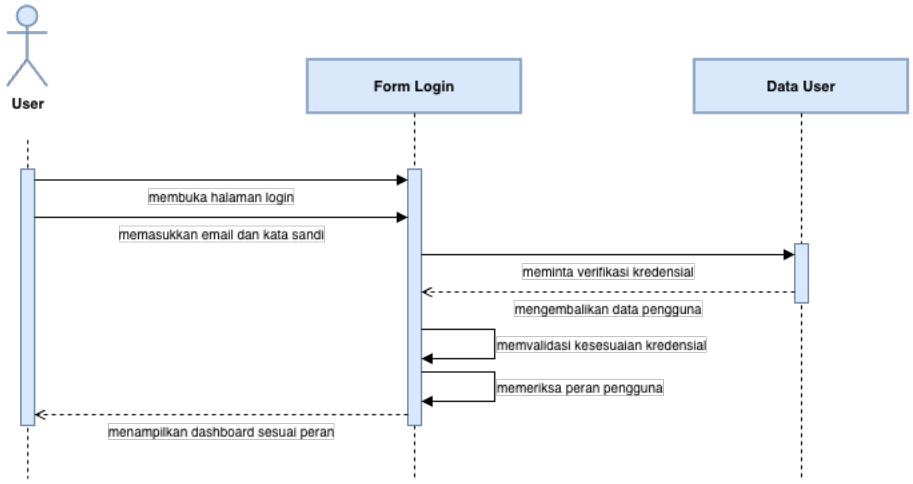
Gambar 4.8 Activity Diagram Pengajuan & Penanganan Eskalasi

4.3.1.3 Model Sequence Diagram

1) *Sequence Diagram* Autentikasi (*Login* dan *Logout)*

Gambar 4.9 memodelkan alur interaksi antarentitas secara kronologis dalam proses autentikasi pengguna pada sistem. Siklus interaksi diinisiasi oleh aktor User yang mengirimkan pesan sinkron (*synchronous message*) ke objek antarmuka Form Login untuk meminta akses halaman dan mendistribusikan input kredensial, yang terdiri dari alamat surel (*email*) dan kata sandi (*password*). Setelah input diterima, objek Form Login meneruskan proses dengan mengirimkan permintaan verifikasi kredensial ke objek Data User yang merepresentasikan lapisan basis data (*database*). Objek Data User kemudian melakukan pencarian rekaman data (*data record*) dan mengirimkan pesan balasan (*return message*) berupa pengembalian data pengguna yang bersangkutan kembali kepada objek antarmuka.

Setelah data diterima dari lapisan basis data, sistem mengeksekusi serangkaian pemrosesan internal komputasi (*self-message*) di dalam Form Login. Pemrosesan internal ini mencakup dua tahapan validasi krusial: pertama, memvalidasi kesesuaian kredensial yang diinputkan dengan data yang tersimpan (proses autentikasi); kedua, memeriksa atribut peran (*role*) dari pengguna tersebut untuk menerapkan hierarki hak akses (*Role-Based Access Control/RBAC*). Sebagai penutup siklus operasional, sistem merespons ke aktor User melalui *return message* dengan merender dan menampilkan antarmuka *dashboard* utama yang secara dinamis disesuaikan dengan peran fungsional pengguna (seperti Manajer atau Teknisi).



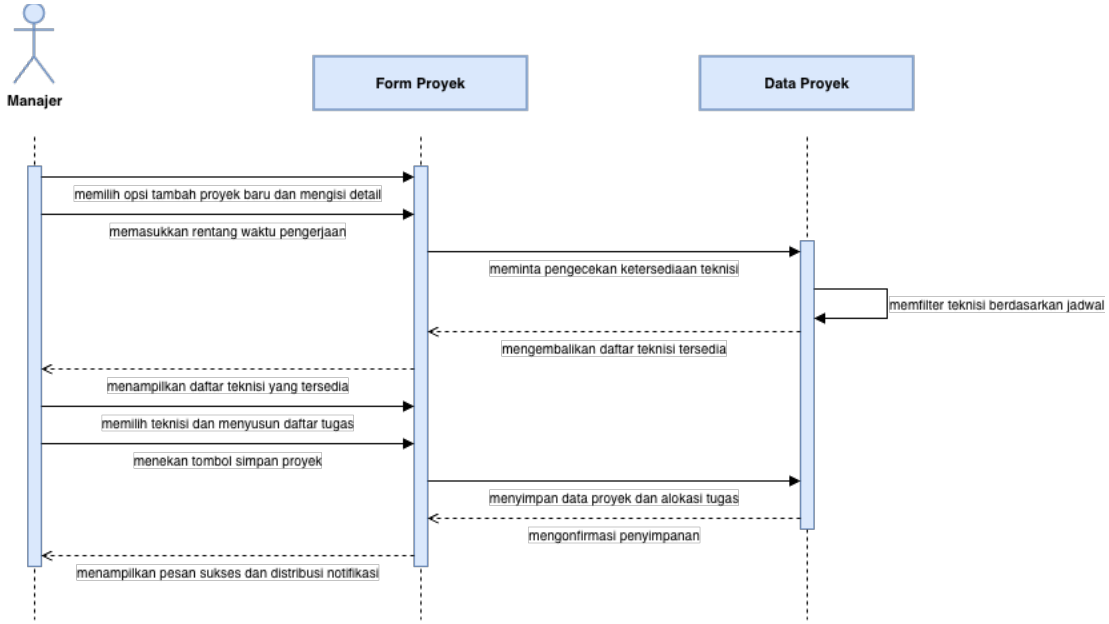
Gambar 4.9 Sequence Diagram Autentikasi

2) *Sequence Diagram* Pengelolaan Proyek

Gambar 4.10 memodelkan alur interaksi antarentitas secara kronologis dalam proses penambahan data proyek baru beserta mekanisme alokasi penugasannya. Siklus komputasi diinisiasi oleh aktor Manajer yang mengirimkan pesan sinkron (*synchronous message*) ke objek antarmuka Form Proyek untuk menginputkan detail spesifikasi proyek dan menentukan rentang waktu kalender pengerjaannya. Input jadwal kerja ini bertindak sebagai pemicu bagi objek Form Proyek untuk meneruskan instruksi permintaan pengecekan ketersediaan sumber daya ke objek lapisan basis data, yaitu Data Proyek. Di dalam objek lapisan data tersebut, terjadi eksekusi pemrosesan internal (*self-message*) berupa kueri penyaringan algoritma untuk memfilter dan mengeliminasi teknisi yang jadwalnya berbenturan. Setelah pemrosesan komputasi selesai, objek Data Proyek mengirimkan pesan balasan (*return message*) berupa pengembalian himpunan data teknisi yang tervalidasi tersedia, yang kemudian dirender oleh Form Proyek untuk disajikan secara visual sebagai rekomendasi kepada Manajer.

Berbekal daftar rekomendasi sistem tersebut, aktor Manajer melanjutkan alur interaksi dengan memilih teknisi yang dialokasikan dan menyusun dekomposisi daftar tugas kerja secara terstruktur. Rangkaian instruksi operasional ini dieksekusi secara

definitif saat Manajer menekan tombol simpan proyek pada antarmuka. Objek Form Proyek kemudian mentransmisikan instruksi penyimpanan kompilasi data proyek beserta relasi alokasi tugasnya kepada objek Data Proyek guna direkam secara permanen (*persistent storage*). Setelah proses penulisan ke basis data berhasil dieksekusi, objek Data Proyek mengirimkan konfirmasi penyimpanan kembali ke antarmuka. Sebagai penutup siklus operasional, objek Form Proyek memberikan umpan balik sistem (*system feedback*) kepada aktor Manajer melalui *return message* dengan menampilkan pesan keberhasilan di layar sekaligus memicu distribusi notifikasi penugasan baru.



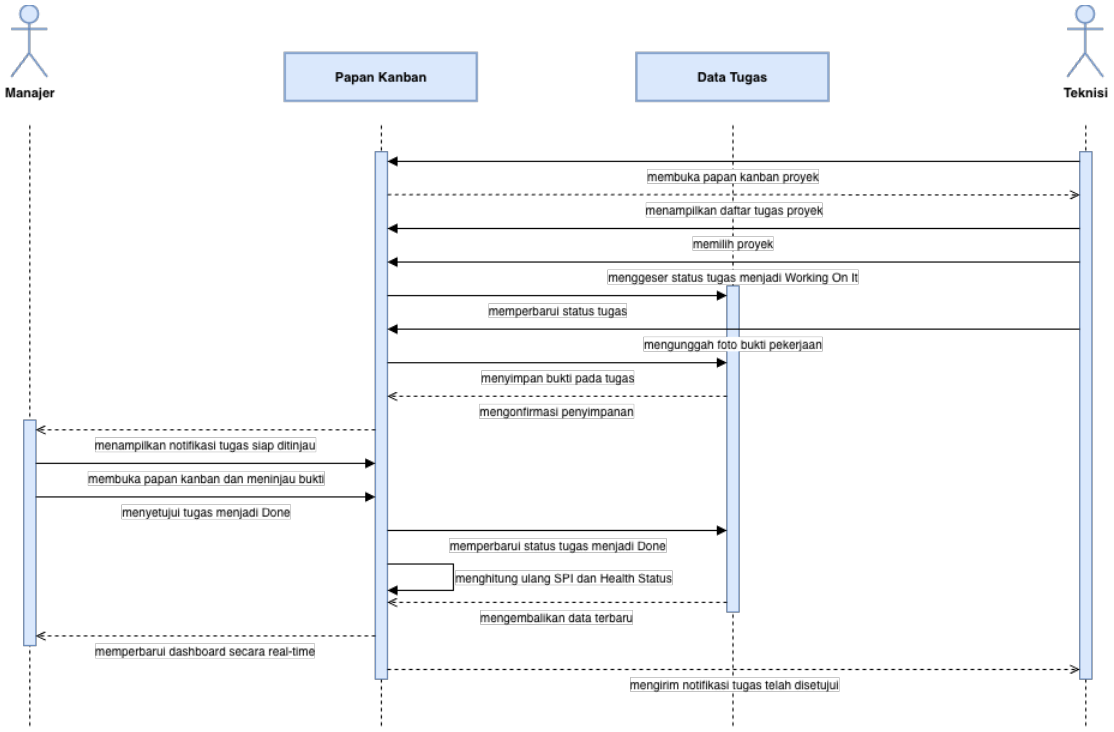
Gambar 4.10 Sequence Diagram Pengelolaan Proyek

3) *Sequence Diagram* Pelaporan Progres Harian, Validasi Tugas, & Kalkulasi SPI (*Review Gate*)

Gambar 4.11 mendeskripsikan alur interaksi sistematis dalam pelaporan progres kerja dan proses validasi penyelesaian tugas. Siklus operasional ini diinisiasi oleh aktor Teknisi yang mengirimkan permintaan akses ke objek antarmuka Papan Kanban untuk membuka dan memilih proyek yang sedang dikerjakan. Setelah sistem merender daftar tugas, Teknisi mengeksekusi instruksi operasional dengan menggeser

indikator status tugas menjadi "*Working On It*" dan mengunggah foto sebagai bukti empiris penyelesaian pekerjaan. Instruksi perubahan ini diteruskan oleh objek Papan Kanban ke objek lapisan basis data, yakni Data Tugas, untuk memproses pembaruan status dan menyimpan *file* bukti pekerjaan secara permanen (*persistent storage*). Setelah proses penulisan (*write operation*) divalidasi oleh basis data, objek Data Tugas mengirimkan pesan konfirmasi (*return message*) kembali ke Papan Kanban. Transaksi ini kemudian secara otomatis memicu sistem untuk mendistribusikan notifikasi lintas aktor kepada Manajer, menginformasikan bahwa terdapat tugas yang telah siap untuk ditinjau (*ready for review*).

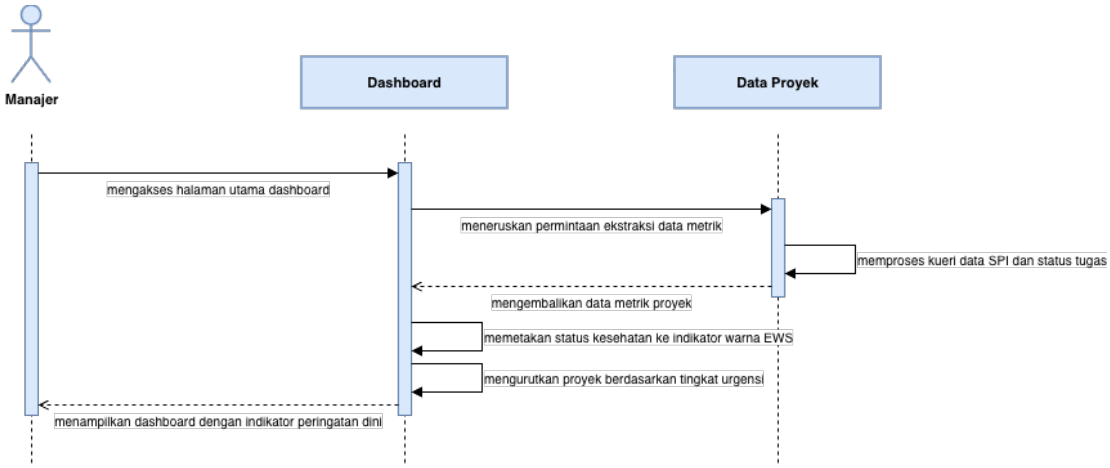
Merespons notifikasi sistem tersebut, aktor Manajer melanjutkan alur interaksi dengan mengakses Papan Kanban guna melakukan peninjauan terhadap bukti pekerjaan yang diunggah oleh Teknisi. Apabila hasil pekerjaan dinyatakan valid, Manajer memberikan otorisasi dengan menyetujui tugas dan menggeser statusnya menjadi "*Done*". Pembaruan status final ini segera ditransmisikan ke objek Data Tugas untuk dicatat. Secara bersamaan, objek Papan Kanban mengeksekusi pemrosesan komputasi algoritmik internal (*self-message*) yang sangat krusial, yaitu menghitung ulang nilai *Schedule Performance Index* (SPI) dan memperbarui *Health Status* proyek secara otomatis berdasarkan penyelesaian tugas tersebut. Siklus komputasi ini mencapai titik akhir (*end state*) ketika objek basis data mengembalikan rekaman entitas terbaru, yang direspons oleh sistem dengan merender pembaruan metrik pada *dashboard* Manajer secara *real-time*, sekaligus mendistribusikan notifikasi penutupan tugas kepada Teknisi.



Gambar 4.11 Sequence Diagram Pelaporan Progres Harian, Validasi Tugas, & Kalkulasi SPI (Review Gate)

4) *Sequence Diagram Dashboard Early Warning System*

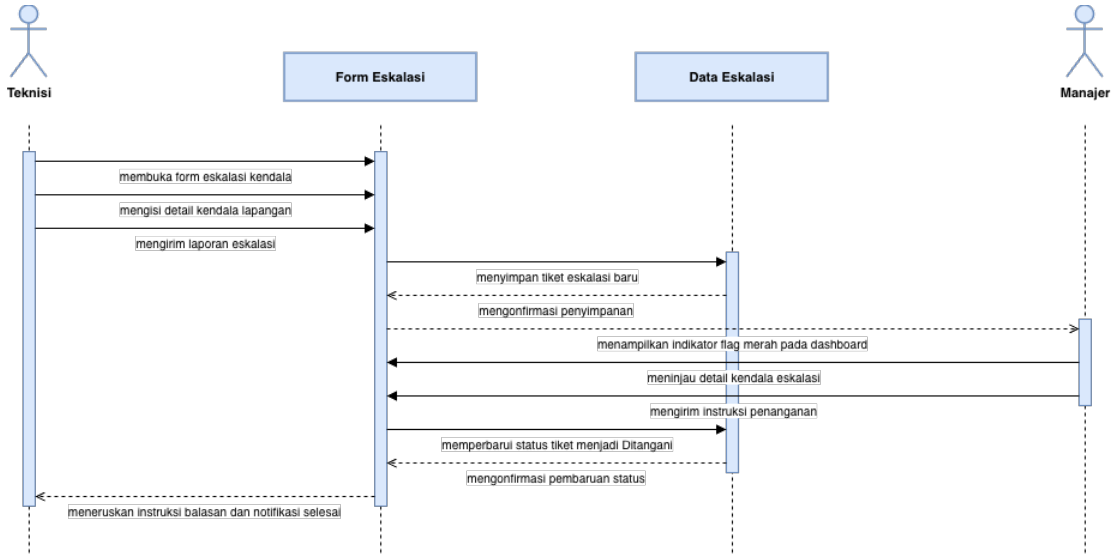
Gambar 4.12 mengilustrasikan alur interaksi sistematis dalam memuat *Dashboard Early Warning System* (EWS). Proses diinisiasi saat aktor Manajer mengakses halaman utama, yang secara otomatis memicu objek Dashboard untuk meminta ekstraksi data metrik kepada lapisan basis data (Data Proyek). Setelah Data Proyek selesai memproses kueri nilai SPI dan status tugas, himpunan data tersebut dikembalikan ke Dashboard. Selanjutnya, objek Dashboard melakukan dua tahapan pemrosesan algoritmik internal: memetakan status kesehatan proyek ke dalam indikator warna EWS secara dinamis, lalu mengurutkan daftar proyek tersebut berdasarkan tingkat urgensi. Alur komputasi ini ditutup dengan Dashboard merender antarmuka visual secara utuh yang menampilkan indikator peringatan dini kepada Manajer.



Gambar 4.12 Sequence Diagram Dashboard Early Warning System

5) *Sequence Diagram* Pengajuan & Penanganan Eskalasi

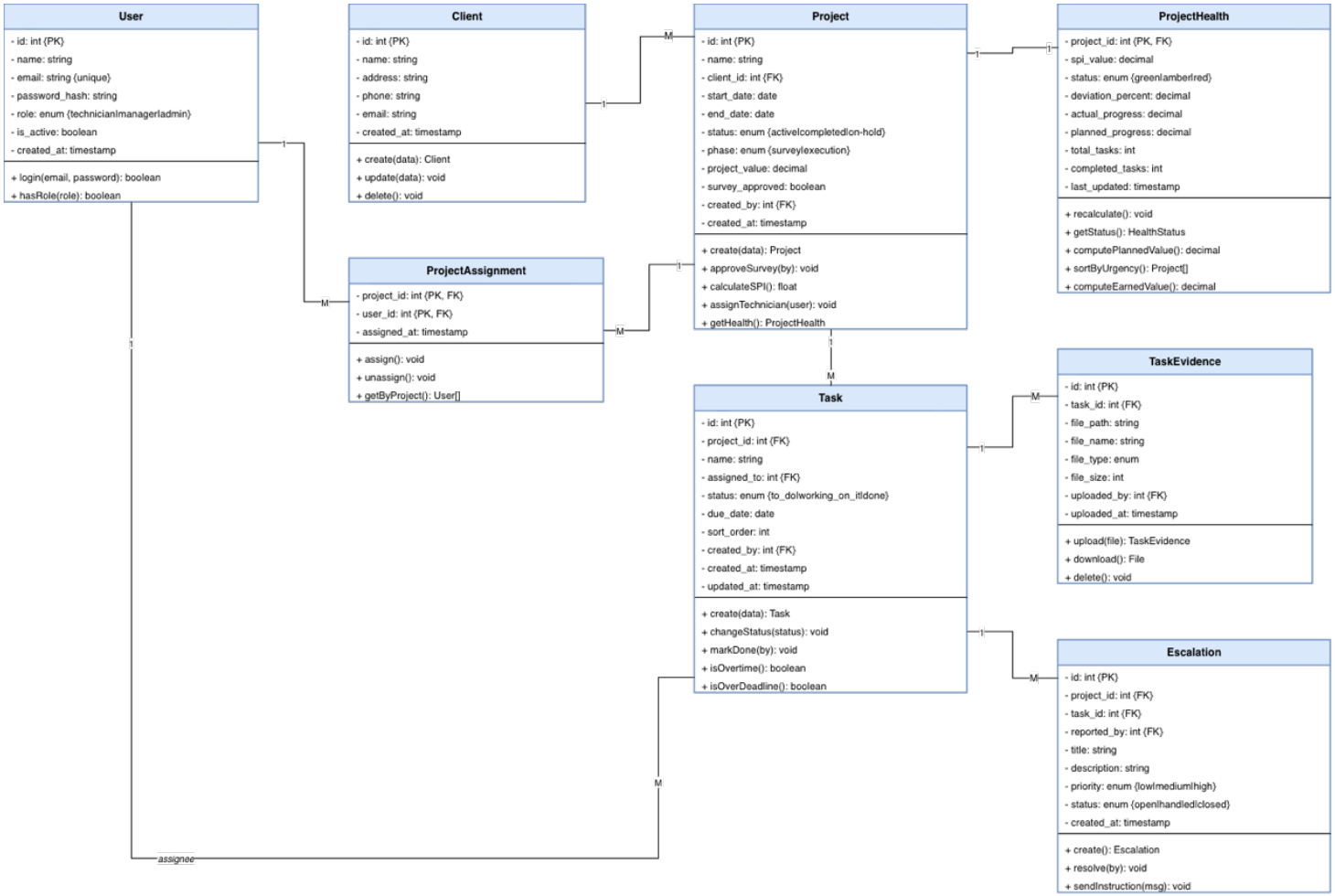
Gambar 4.13 mengilustrasikan alur komunikasi sistematis dua arah dalam proses pelaporan dan penanganan eskalasi kendala operasional. Proses ini diinisiasi oleh aktor Teknisi yang mengirimkan rincian kendala lapangan melalui antarmuka Form Eskalasi, untuk kemudian direkam secara permanen oleh lapisan basis data (Data Eskalasi). Sistem secara proaktif mendistribusikan peringatan berupa indikator *flag* merah ke *dashboard* Manajer guna memicu atensi visual secara *real-time*. Sebagai respons, Manajer meninjau tiket eskalasi tersebut dan memberikan instruksi penanganan sebagai solusi. Siklus komputasi ini ditutup dengan pembaruan status tiket menjadi "Ditangani" di basis data, yang secara otomatis memicu antarmuka sistem untuk meneruskan instruksi balasan dari Manajer beserta notifikasi penyelesaian kembali kepada Teknisi.



Gambar 4.13 Sequence Diagram Pengajuan & Penanganan Eskalasi

4.3.1.4 Model Class Diagram

Class diagram merupakan salah satu instrumen pemodelan dalam *Unified Modeling Language* (UML) yang berfungsi untuk mendeskripsikan struktur statis sebuah sistem dengan menjabarkan entitas kelas, atribut, metode, serta arsitektur relasi antarobjek di dalamnya. Diagram ini merepresentasikan implementasi kode program yang membangun logika komputasi pada sistem *project management*. Mengingat sistem ini difokuskan pada pengembangan fitur *dashboard* analitik yang bersumber dari data *daily report*, maka arsitektur kelas-kelas utama dirancang secara spesifik untuk mengakomodasi hierarki manajemen pengguna, pengelolaan entitas proyek, perekaman data pelaporan tugas harian dari lapangan, hingga pemrosesan logika metrik *Early Warning System* (EWS). *Class diagram* dapat dilihat pada Gambar 4.14



Gambar 4.14 Class Diagram

.

Sebagai penjabaran lebih lanjut dari rancangan *Class Diagram* yang telah diuraikan sebelumnya, berikut adalah pemaparan mengenai fungsi spesifik dari masing-masing kelas utama:

1. Class User

Kelas User berfungsi sebagai lapisan autentikasi dan otorisasi pada sistem. Kelas ini merepresentasikan entitas pengguna serta mengelola hak akses berdasarkan peran pengguna, seperti Manajer, Teknisi, dan Admin. Deskripsi fungsi pada kelas User dapat dilihat pada Tabel 4.1.

Tabel 4.1 Class User

No.	Fungsi	Deskripsi
1.	*login*	Memvalidasi kredensial berupa *email* dan kata sandi untuk memberikan akses masuk ke dalam sistem.
2.	*hasRole*	Memeriksa dan memastikan hak akses atau peran pengguna dalam sistem.

2. Class Client

Kelas *Client* merepresentasikan entitas pelanggan atau instansi pemilik proyek. Kelas ini digunakan untuk mengelola data klien yang terdaftar dalam sistem. Deskripsi fungsi pada kelas *Client* dapat dilihat pada Tabel 4.2.

Tabel 4.2 Class Client

No.	Fungsi	Deskripsi
1.	*create*	Menambahkan dan menyimpan data profil klien baru ke dalam basis data.
2.	*update*	Memperbarui informasi klien yang telah terdaftar dalam sistem.
3.	*delete*	Menghapus data klien dari sistem.

3. Class Project

Kelas *Project* merupakan inti utama sistem karena berfungsi untuk mengelola koordinasi proyek, penjadwalan, fase pekerjaan, serta hubungan antar tugas. Deskripsi fungsi pada kelas *Project* dapat dilihat pada Tabel 4.3.

Tabel 4.3 Class Project

No.	Fungsi	Deskripsi
1.	*create*	Membuat dan merekam data proyek baru ke dalam sistem.
2.	*approveSurvey*	Mengubah status fase proyek setelah proses survei lapangan disetujui.
3.	*calculateSPI*	Melakukan perhitungan nilai *Schedule Performance Index* (SPI) pada proyek.
4.	*assignTechnician*	Mengalokasikan teknisi ke dalam operasional proyek.
5.	*getHealth*	Mengambil informasi status kesehatan proyek dari kelas *ProjectHealth*.

4. Class ProjectAssignment

Kelas *ProjectAssignment* berfungsi sebagai kelas asosiasi (*association class*) yang menghubungkan pengguna dengan proyek tertentu secara terstruktur. Deskripsi fungsi pada kelas *ProjectAssignment* dapat dilihat pada Tabel 4.4.

Tabel 4.4 Class ProjectAssignment

No.	Fungsi	Deskripsi
1.	*assign*	Menetapkan relasi penugasan pengguna ke dalam suatu proyek.
2.	*unassign*	Menghapus relasi penugasan pengguna dari proyek.
3.	*getByProject*	Mengambil daftar pengguna atau teknisi yang ditugaskan pada proyek tertentu.

5. Class ProjectHealth

Kelas *ProjectHealth* berfungsi untuk mengelola proses komputasi *Early Warning System* (EWS) serta menghitung indikator kesehatan proyek. Dengan adanya pemisahan ini, proses analisis tidak membebani kelas utama *Project*. Deskripsi fungsi pada kelas *ProjectHealth* dapat dilihat pada Tabel 4.5.

Tabel 4.5 Class ProjectHealth

No.	Fungsi	Deskripsi
1.	*recalculate*	Menghitung ulang persentase deviasi dan metrik proyek berdasarkan data terbaru.
2.	*getStatus*	Mengonversi hasil perhitungan metrik menjadi indikator status peringatan, seperti hijau, kuning, atau merah.
3.	*computePlannedValue*	Menghitung nilai progres rencana berdasarkan jadwal proyek.
4.	*sortByUrgency*	Mengurutkan prioritas tampilan proyek pada *dashboard* berdasarkan tingkat urgensi.
5.	*computeEarnedValue*	Menghitung nilai progres aktual berdasarkan tugas yang telah selesai.

6. Class Task

Kelas *Task* merepresentasikan unit pekerjaan atau tugas lapangan yang harus diselesaikan dalam periode waktu tertentu. Deskripsi fungsi pada kelas *Task* dapat dilihat pada Tabel 4.6.

Tabel 4.6 Class Task

No.	Fungsi	Deskripsi
1.	*create*	Membuat rincian tugas baru dalam suatu proyek.
2.	*changeStatus*	Memperbarui status pengerjaan tugas, seperti *to_do*, *working_on*, atau *done*.
3.	*markDone*	Menandai bahwa tugas telah selesai secara permanen.
4.	*isOvertime*	Mengevaluasi apakah durasi pengerjaan melebihi estimasi waktu yang ditentukan.
5.	*isOverDeadline*	Mengidentifikasi tugas yang melewati batas waktu penyelesaian (*due date*).

7. Class TaskEvidence

Kelas *TaskEvidence* digunakan untuk mengelola dokumen atau lampiran bukti pekerjaan yang dikirimkan oleh teknisi. Deskripsi fungsi pada kelas *TaskEvidence* dapat dilihat pada Tabel 4.7.

Tabel 4.7 Class TaskEvidence

No.	Fungsi	Deskripsi
1.	*upload*	Menyimpan berkas gambar atau dokumen bukti pekerjaan ke dalam sistem.
2.	*download*	Mengambil berkas bukti pekerjaan untuk ditinjau atau diunduh.
3.	*delete*	Menghapus berkas bukti pekerjaan dari sistem penyimpanan.

8. Class Escalation

Kelas *Escalation* berfungsi untuk mencatat, mengelola prioritas, dan memantau proses komunikasi terkait kendala lapangan yang terjadi selama pelaksanaan proyek. Deskripsi fungsi pada kelas *Escalation* dapat dilihat pada Tabel 4.8.

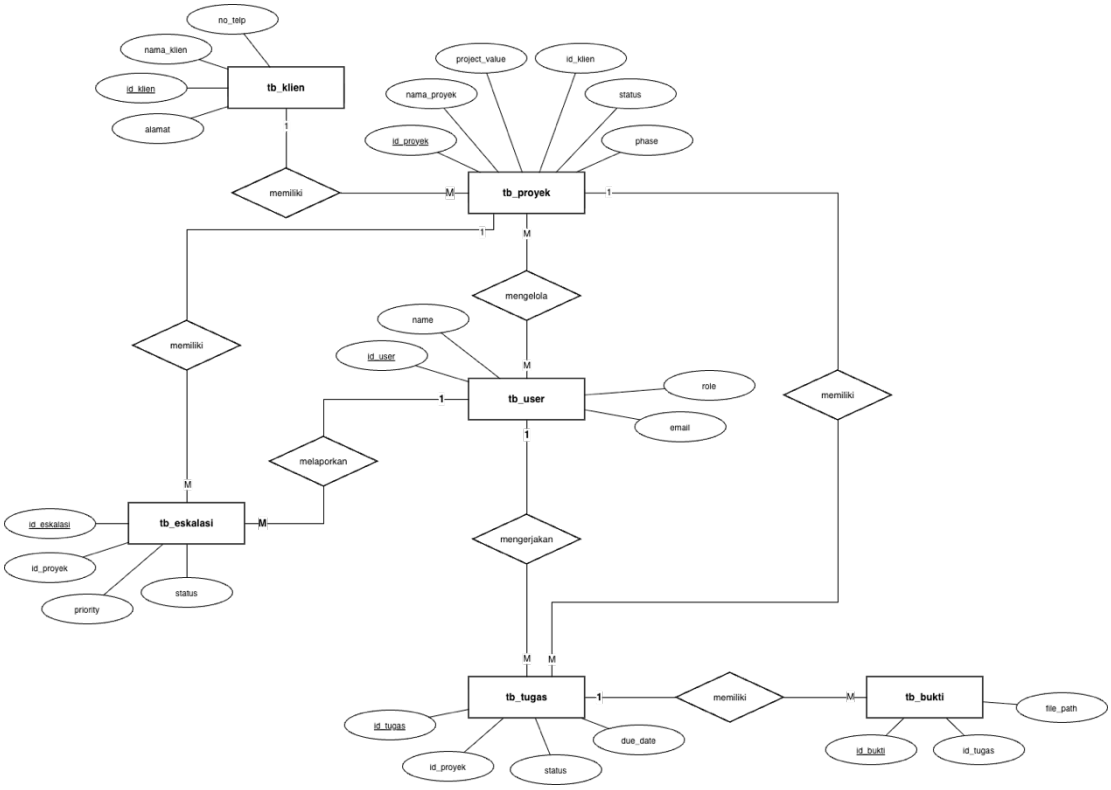
Tabel 4.8 Class Escalation

No.	Fungsi	Deskripsi
1.	*create*	Mencatat laporan eskalasi atau kendala baru yang dikirimkan oleh teknisi.
2.	*resolve*	Memperbarui status kendala menjadi selesai atau telah ditangani.
3.	*sendInstruction*	Mengirimkan instruksi atau solusi dari manajer kepada teknisi pelapor.

4.3.2 Perancangan Model Data

Dalam pengembangan fitur *dashboard* pada sistem *project management* berdasarkan data *daily report* untuk PT Smart Home Inovasi (SHI), penerapan *Entity Relationship Diagram* (ERD) menjadi fondasi utama dalam membangun struktur basis data yang solid. Diagram ini memetakan hubungan relasional antara entitas-entitas vital dalam sistem, meliputi Klien, Proyek, User, Tugas, Bukti, dan Eskalasi. Struktur data ini dirancang secara spesifik untuk mengakomodasi alur kerja operasional, mulai dari inisiasi proyek klien, manajemen peran pengguna, dekomposisi pekerjaan lapangan, pengumpulan data pelaporan harian (*daily report*), hingga penanganan kendala sistematis. Melalui implementasi desain ERD ini, sistem mampu mensentralisasi data operasional lapangan, menjamin integritas referensi antar data, serta menyuplai

kompilasi data yang akurat secara *real-time* untuk dikomputasi menjadi metrik analitik pada *dashboard* pemantauan.



Gambar 4.15 Entity Relationship Diagram

Entitas dan Hubungannya:

1) Tabel Klien (tb_klien)

Tabel ini menyimpan data fundamental mengenai entitas pelanggan atau pemilik proyek, mencakup informasi seperti ID klien, nama klien, nomor telepon, dan alamat. Setiap klien dapat memiliki banyak proyek yang terdaftar di dalam sistem, yang tercermin dari hubungan satu-ke-banyak (1:N) melalui relasi "memiliki" antara tb_klien dan tb_proyek. Artinya, satu klien dapat memiliki lebih dari satu proyek, namun setiap proyek secara eksklusif hanya dimiliki oleh satu klien.

2) Tabel Proyek (tb_proyek)

Tabel ini menyimpan informasi inti terkait pelaksanaan proyek operasional, seperti ID proyek, nama proyek, nilai proyek (*project value*), fase (*phase*), status, serta kunci tamu (ID klien). Sebagai pusat operasional, setiap entitas proyek menjadi induk dari berbagai rincian kerja. Tabel ini memiliki hubungan satu-ke-banyak (1:N) dengan tb_tugas dan tb_eskalasi, yang berarti satu proyek dapat memiliki banyak rincian tugas serta laporan eskalasi kendala.

3) Tabel User (tb_user)

Tabel ini berisi data kredensial dan identitas mengenai pengguna sistem, yang mencakup ID user, nama, email, dan peran fungsional (*role* seperti Manajer atau Teknisi). Relasi banyak-ke-banyak (M:N) antara tb_user dan tb_proyek melalui relasi "mengelola" menggambarkan bahwa pengguna dengan hak akses manajerial dapat mengelola banyak proyek secara bersamaan, dan satu proyek dapat dikelola oleh banyak manajer. Selain itu, terdapat relasi satu-ke-banyak (1:N) terhadap tb_tugas (melalui relasi "mengerjakan") dan tb_eskalasi (melalui relasi "melaporkan"), yang menunjukkan bahwa satu teknisi dapat mengerjakan banyak rincian tugas dan melaporkan berbagai eskalasi dari lapangan.

4) Tabel Tugas (tb_tugas)

Tabel ini berfungsi sebagai wadah utama untuk pencatatan *daily report* teknisi. Entitas ini menyimpan informasi mengenai rincian pekerjaan lapangan, mencakup ID tugas, ID proyek, status pengerjaan, dan tenggat waktu (*due date*). Setiap entitas tugas terhubung dengan satu proyek dan dikerjakan oleh satu teknisi. Untuk mengakomodasi pelaporan visual, tabel ini memiliki relasi satu-ke-banyak (1:N) dengan tb_bukti, yang berarti satu tugas pekerjaan dapat memiliki banyak *file* bukti lampiran yang diunggah oleh teknisi.

5) Tabel Bukti (tb_bukti)

Tabel ini menyimpan *path* atau direktori lokasi dari berkas lampiran visual yang membuktikan progres penyelesaian pekerjaan di lapangan. Atributnya mencakup ID bukti, ID tugas, dan lokasi *file* (*file path*). Keberadaan tabel ini secara spesifik

memfasilitasi arsip pelaporan *daily report* agar terpusat dan mudah diverifikasi oleh manajer melalui *dashboard*.

6) Tabel Eskalasi (tb_eskalasi)

Tabel ini menyimpan data mengenai pelaporan kendala atau masalah yang terjadi selama operasional proyek berlangsung, mencakup atribut ID eskalasi, ID proyek, tingkat prioritas (*priority*), dan status penanganan. Setiap tiket eskalasi dilaporkan oleh satu teknisi (terhubung ke tb_user) dan merujuk pada satu proyek spesifik (terhubung ke tb_proyek). Struktur ini memastikan setiap hambatan di lapangan terekam dengan jelas dan dapat segera ditindaklanjuti oleh manajemen.

4.3.3 Perancangan Fisik Basis Data

4.3.4 Perancangan Relasi Antar Data

4.3.5 Perancangan Antarmuka

4.3.5.1 Antarmuka Data Input

1) Halaman Login
2) Halaman Tambah Proyek
3) Halaman Tambah Daily Report

4.3.5.2 Antarmuka Data Proses

4.3.5.3 Antarmuka Data *Output*

4.3.5.4 Antarmuka Navigasi Struktur Data

4.4 Rancangan Anggaran Pengembangan Sistem

BAB V

IMPLEMENTASI DAN PEMBAHASAN SISTEM

5.1 Implementasi

5.1.1 Implementasi Basis Data

5.1.2 ‘Implementasi Sistem

5.2 Pembahasan

5.2.1 Pembahasan Basis Data

5.2.2 Pembahasan Sistem

5.2.3 Pengujian

5.3 Inovasi Sistem

f

BAB VI
PENUTUP

6.1 Simpulan

6.2 Saran

DAFTAR PUSTAKA

LAMPIRAN

LOGBOOK TUGAS AKHIR